

`

Rapport de projet

5ème année

Ingénierie Informatique et Réseaux

Sous le thème

SPRING_CommerceFlow-MS

Conception et Développement d'une Plateforme E-Commerce Basée sur une Architecture Microservices

Réalisé par :

MAJJID Ayoub

EL HILALI Ayman

Encadré par :

Tuteur de l'école : Prof. Hatim JAADOUNI

Année Universitaire : 2025-2026

DÉDICACE

À nos familles,

Pour leur soutien inconditionnel, leur amour et leur encouragement constants tout au long de ce parcours académique.

À nos parents,

Qui ont toujours cru en nous et nous ont encouragés à poursuivre nos rêves.

À nos professeurs et mentors,

Pour leur sagesse et leurs conseils précieux.

REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Nous remercions sincèrement **Prof. Hatim JAADOUNI**, notre encadrant, pour son expertise, ses conseils avisés et sa disponibilité tout au long de ce projet.

Nous remercions également l'**École Marocaine des Sciences de l'Ingénieur (EMSI)** et tout le corps professoral pour la qualité de l'enseignement dispensé.

Enfin, nous exprimons notre reconnaissance à nos familles pour leur soutien inconditionnel.

RÉSUMÉ

Ce rapport présente notre projet de fin d'année portant sur la conception et le développement d'une plateforme e-commerce basée sur une **architecture microservices** utilisant **Spring Boot** et **Spring Cloud**.

Dans un contexte où les applications monolithiques montrent leurs limites en termes de scalabilité et de maintenabilité, l'adoption d'une architecture microservices s'impose comme solution de référence pour les entreprises modernes.

Note : Ce projet est actuellement **en cours de développement**. Les composants principaux sont fonctionnels, mais certaines fonctionnalités avancées sont encore en phase d'implémentation.

Le système en développement comprend **4 microservices** (Product, Order, Inventory, Notification), une **API Gateway centralisée**, une **persistance polyglotte** (MongoDB + MySQL), et une **sécurité OAuth2/OIDC** avec Keycloak.

Mots-clés : Microservices, Spring Boot, Spring Cloud, API Gateway, Keycloak, Docker, OAuth2, JWT

ABSTRACT

This report presents our final year project focusing on the design and development of an e-commerce platform based on a **microservices architecture** using **Spring Boot** and **Spring Cloud**.

In a context where monolithic applications show their limitations in terms of scalability and maintainability, adopting a microservices architecture becomes the reference solution for modern enterprises.

Note: This project is currently **under active development**. Core components are functional, while advanced features are being implemented.

The system under development includes **4 microservices** (Product, Order, Inventory, Notification), a **centralized API Gateway**, **polyglot persistence** (MongoDB + MySQL), and **OAuth2/OIDC security** with Keycloak.

Keywords: Microservices, Spring Boot, Spring Cloud, API Gateway, Keycloak, Docker, OAuth2, JWT

TABLE DES MATIÈRES

- 1. [Introduction et Contexte](#)
- 2. [Revue de Littérature et Contexte Technologique](#)
- 3. [Analyse et Expression des Besoins](#)
- 4. [Conception du Système](#)
- 5. [Réalisation et Implémentation](#)
- 6. [Tests et Validation](#)
- 7. [Résultats et Discussion](#)
- 8. [Conclusion et Perspectives](#)
- 9. [Webographie](#)
- 10. [Annexes](#)

LISTE DES ABRÉVIATIONS

Abréviation	Signification
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
DTO	Data Transfer Object
E2E	End-to-End
HTTP	HyperText Transfer Protocol
IAM	Identity and Access Management
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
NoSQL	Not Only SQL
OAuth	Open Authorization

Abréviation	Signification
OIDC	OpenID Connect
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SOA	Service-Oriented Architecture
SQL	Structured Query Language
SSO	Single Sign-On

CHAPITRE 1

INTRODUCTION ET CONTEXTE

1.1 Introduction Générale

Dans le paysage technologique actuel, les exigences des applications d'entreprise ont considérablement évolué. Les utilisateurs attendent des systèmes **performants, disponibles 24/7**, et capables de **s'adapter rapidement** aux changements du marché. Face à ces défis, l'architecture **microservices** s'est imposée comme une solution de référence, permettant de construire des applications sous forme de **services autonomes**, faiblement couplés et indépendamment déployables. Ce projet s'inscrit dans cette démarche en proposant la conception et le développement d'une plateforme e-commerce moderne basée sur l'écosystème **Spring Boot** et **Spring Cloud**.

1.2 Problématique

Les architectures monolithiques traditionnelles présentent plusieurs limitations critiques :

Problème	Impact sur l'Entreprise
Couplage fort	Une modification nécessite le redéploiement complet de l'application
Scalabilité limitée	Impossible de scaler un seul composant indépendamment
Technologie unique	Obligation d'utiliser le même langage pour toute l'application
Déploiement risqué	Un bug dans un module affecte l'ensemble du système

Question Centrale : Comment concevoir un système distribué capable de gérer les opérations e-commerce (produits, commandes, inventaire) de manière **scalable, résiliente** et **sécurisée**, tout en maintenant une **indépendance technologique** entre les composants ?

1.3 Objectifs du Projet

Catégorie	Objectif	Métrique de Succès
Architecture	Concevoir une architecture microservices robuste	4+ services indépendants
Développement	Implémenter les services métier	CRUD complet par service
Intégration	Établir la communication inter-services	OpenFeign fonctionnel
Sécurité	Mettre en place OAuth2/OIDC	Keycloak intégré
Qualité	Assurer la couverture de tests	> 70% coverage

1.4 Périmètre du Projet

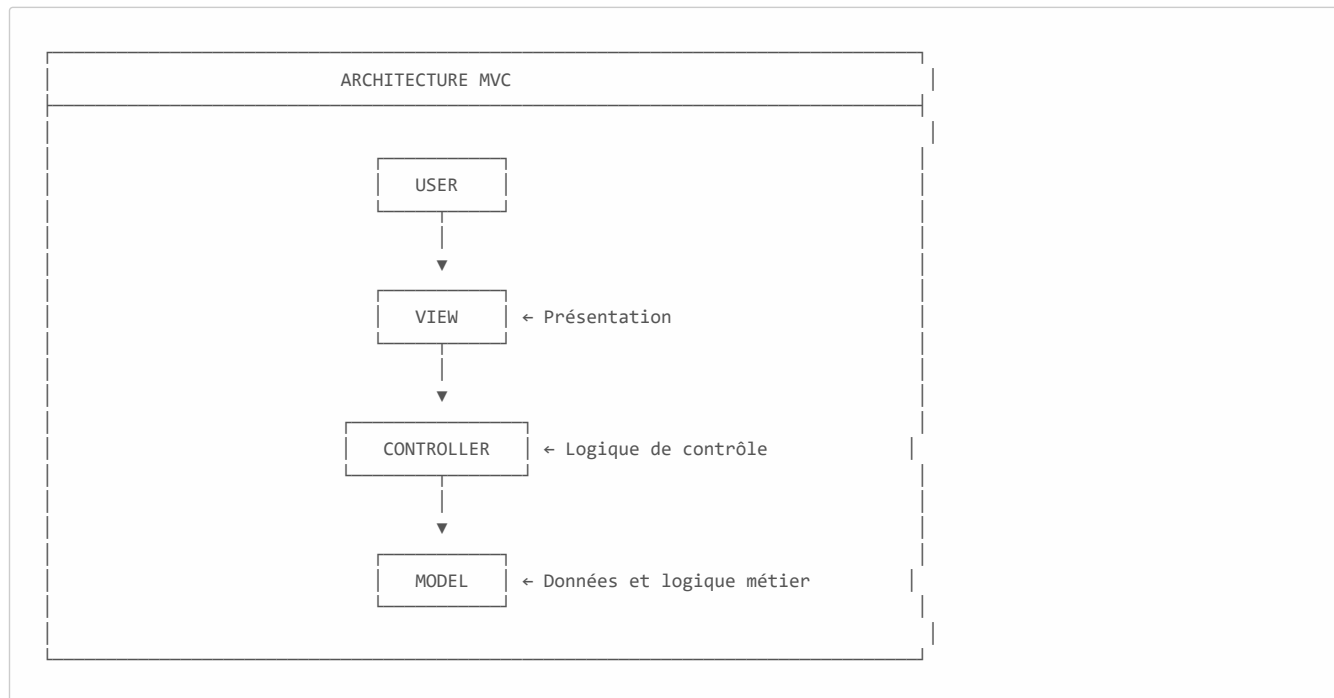
- Scaling vertical uniquement

Limitations :

- Temps de démarrage long
- Déploiement risqué
- Difficile à maintenir à grande échelle

2.1.3 Architecture MVC

Le pattern **Model-View-Controller** a apporté une première forme de séparation des préoccupations.



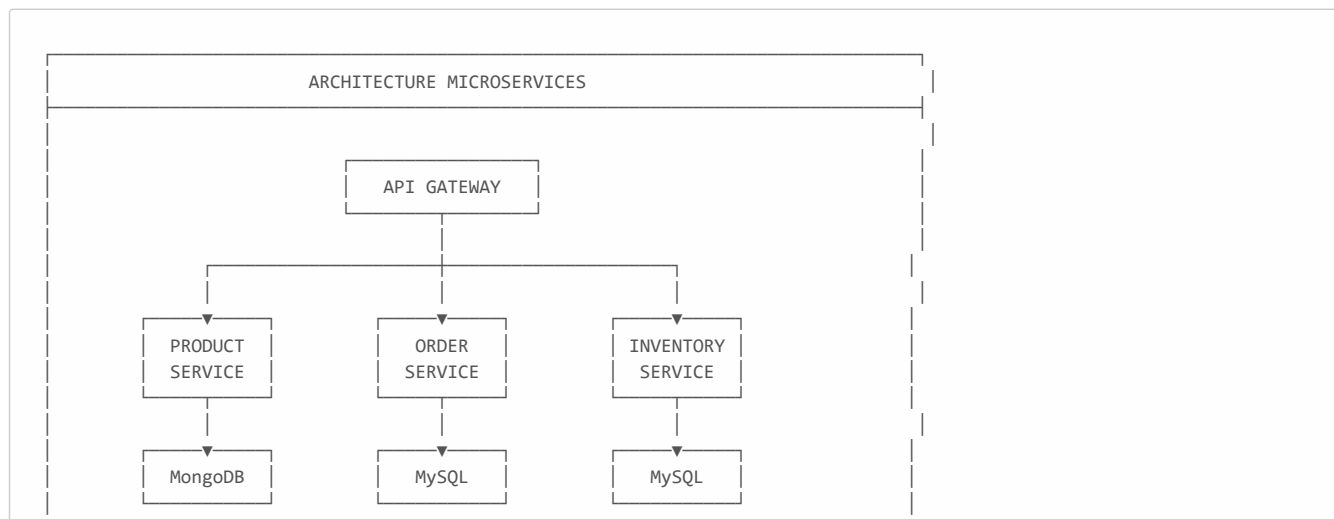
2.1.4 Architecture Orientée Services (SOA)

SOA a introduit la notion de services réutilisables communiquant via des protocoles standards.

Aspect	SOA	Microservices
Couplage	Moyenne	Faible
Granularité	Services larges	Services fins
Communication	ESB centralisé	API REST directe
Base de données	Partagée	Par service

2.1.5 Architecture Microservices

L'architecture microservices décompose l'application en **services autonomes**, chacun responsable d'une fonctionnalité métier spécifique.



- ☒ Chaque service a sa propre base de données (Polyglot Persistence)
- ☒ Communication via REST API ou Messaging
- ☒ Déploiement indépendant

Principes Clés :

1. **Single Responsibility** : Un service = une fonctionnalité métier
2. **Autonomie** : Chaque service peut être développé/déployé indépendamment
3. **Décentralisation** : Pas de point central de contrôle
4. **Résilience** : La défaillance d'un service n'affecte pas les autres

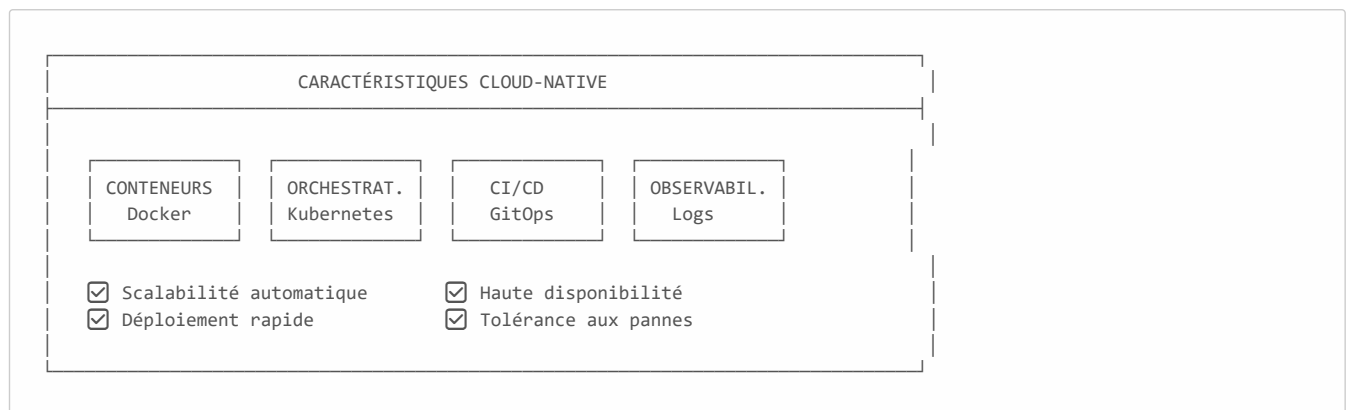
2.1.6 Tableau Comparatif

Critère	Monolithique	SOA	Microservices
Déploiement	Tout ensemble	Par groupe	Par service
Scaling	Vertical	Horizontal limité	Horizontal granulaire
Technologie	Unique	Variée limitée	Polyglotte
Équipes	Centralisée	Par domaine	Par service
Complexité	Simple au début	Moyenne	Élevée (infrastructure)

2.2 Concepts Cloud-Native

2.2.1 Définition

Une application **Cloud-Native** est conçue spécifiquement pour tirer parti des environnements cloud modernes.



2.2.2 Conteneurisation avec Docker

Docker permet de packager une application avec toutes ses dépendances dans un **conteneur** isolé.

Avantages :

- **Portabilité** : "Build once, run anywhere"
- **Isolation** : Chaque service dans son environnement
- **Légèreté** : Partage du kernel de l'hôte

2.2.3 Orchestration avec Kubernetes

Kubernetes gère le déploiement, la mise à l'échelle et la gestion des applications conteneurisées.

Composant	Rôle
Pod	Plus petite unité déployable
Service	Exposition réseau stable
Deployment	Gestion du cycle de vie
Ingress	Routage HTTP externe

2.3 L'Écosystème Spring

2.3.1 Spring Boot



Spring Boot simplifie la création d'applications Spring production-ready.

Caractéristiques :

- Configuration automatique (Auto-configuration)
- Serveur embarqué (Tomcat)
- Actuators pour le monitoring
- Profils d'environnement

2.3.2 Spring Cloud



Spring Cloud fournit les outils pour construire des systèmes distribués.

Composant	Fonction
Spring Cloud Gateway	API Gateway et routage
Spring Cloud OpenFeign	Client HTTP déclaratif
Spring Cloud Config	Configuration centralisée
Eureka	Service Discovery

2.4 Sécurité dans les Microservices

2.4.1 Défis de la Sécurité Distribuée

Dans une architecture microservices, la sécurité doit être gérée de manière **centralisée** mais **appliquée** de manière **distribuée**.

2.4.2 OAuth 2.0 et OpenID Connect

Protocole	Rôle
OAuth 2.0	Autorisation (accès aux ressources)
OpenID Connect	Authentification (identité de l'utilisateur)

2.4.3 Keycloak



Keycloak est une solution IAM open-source développée par Red Hat.

Fonctionnalités :

- Single Sign-On (SSO)
- Identity Brokering (Google, GitHub, LDAP)
- Gestion des utilisateurs et rôles
- Tokens JWT

2.5 Conclusion du Chapitre

Ce chapitre a présenté l'évolution des architectures logicielles, des systèmes monolithiques aux approches cloud-native. Les microservices, combinés à Spring Boot et Spring Cloud, offrent une solution moderne pour construire des applications scalables et maintenables.

CHAPITRE 3

ANALYSE ET EXPRESSION DES BESOINS

3.1 Analyse de l'Existant

Dans le contexte e-commerce traditionnel, les solutions monolithiques présentent les limitations suivantes :

Problème Identifié	Impact Métier
Indisponibilité lors des mises à jour	Perte de revenus
Scalabilité limitée pendant les pics	Mauvaise expérience utilisateur
Couplage des équipes de développement	Cycle de release rallongé
Single point of failure	Risque d'arrêt total

3.2 Besoins Fonctionnels

3.2.1 Gestion des Produits (Product Service)

ID	Exigence	Priorité	Statut
FR-P-01	Créer un nouveau produit	Haute	☒ Implémenté
FR-P-02	Consulter les détails d'un produit	Haute	☒ Implémenté
FR-P-03	Mettre à jour les informations produit	Moyenne	☒ Implémenté
FR-P-04	Supprimer un produit	Moyenne	☒ Implémenté
FR-P-05	Lister tous les produits	Haute	☒ Implémenté

3.2.2 Gestion des Commandes (Order Service)

ID	Exigence	Priorité	Statut
FR-O-01	Passer une nouvelle commande	Haute	☒ Implémenté
FR-O-02	Valider la disponibilité avant commande	Haute	☒ Implémenté
FR-O-03	Annuler une commande	Haute	☒ Implémenté
FR-O-04	Consulter les détails d'une commande	Haute	☒ Implémenté
FR-O-05	Lister toutes les commandes	Moyenne	☒ Implémenté

3.2.3 Gestion de l'Inventaire (Inventory Service)

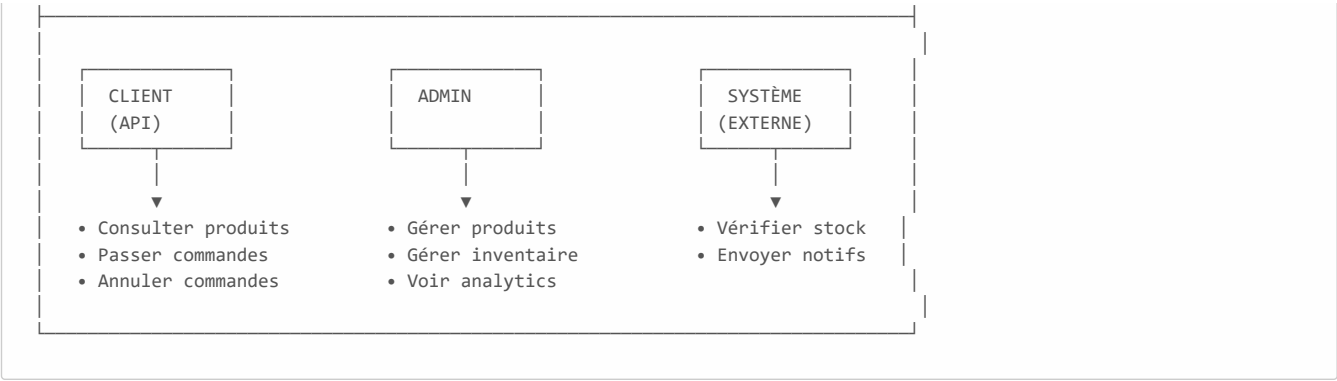
ID	Exigence	Priorité	Statut
FR-I-01	Vérifier la disponibilité en stock	Haute	☒ Implémenté
FR-I-02	Diminuer le stock (vente)	Haute	☒ Implémenté
FR-I-03	Augmenter le stock (achat)	Haute	☒ Implémenté
FR-I-04	Créer un enregistrement d'inventaire	Moyenne	☒ Implémenté
FR-I-05	Mettre à jour l'inventaire	Moyenne	☒ Implémenté

3.3 Besoins Non-Fonctionnels

Catégorie	Exigence	Cible
Performance	Temps de réponse API	< 200ms (95ème percentile)
Scalabilité	Utilisateurs concurrents	1000+
Disponibilité	Uptime système	99.9%
Fiabilité	Taux d'erreur	< 0.1%
Maintenabilité	Couverture de tests	> 70%
Sécurité	Chiffrement	TLS 1.3

3.4 Acteurs du Système





3.5 Contraintes du Projet

Type	Contrainte
Technique	Utilisation de Spring Boot 4.0 et Java 21
Technique	Persistance polyglotte (MongoDB + MySQL)
Temporelle	Livraison en Janvier 2025
Budget	Outils open-source uniquement

3.6 Conclusion du Chapitre

L'analyse des besoins a permis d'identifier 15 exigences fonctionnelles réparties entre les trois services métier principaux, ainsi que des critères de performance et de disponibilité stricts.

CHAPITRE 4

CONCEPTION DU SYSTÈME

4.1 Architecture Générale

4.1.1 Vue d'Ensemble

Notre plateforme **SPRING_CommerceFlow-MS** est conçue selon une architecture microservices complète intégrant tous les composants nécessaires à un système distribué moderne.

 **Lien du Projet Portfolio :**
<https://majjid.com/project.html?project=#spring-commerceflow-ms>

4.1.2 Diagramme d'Architecture

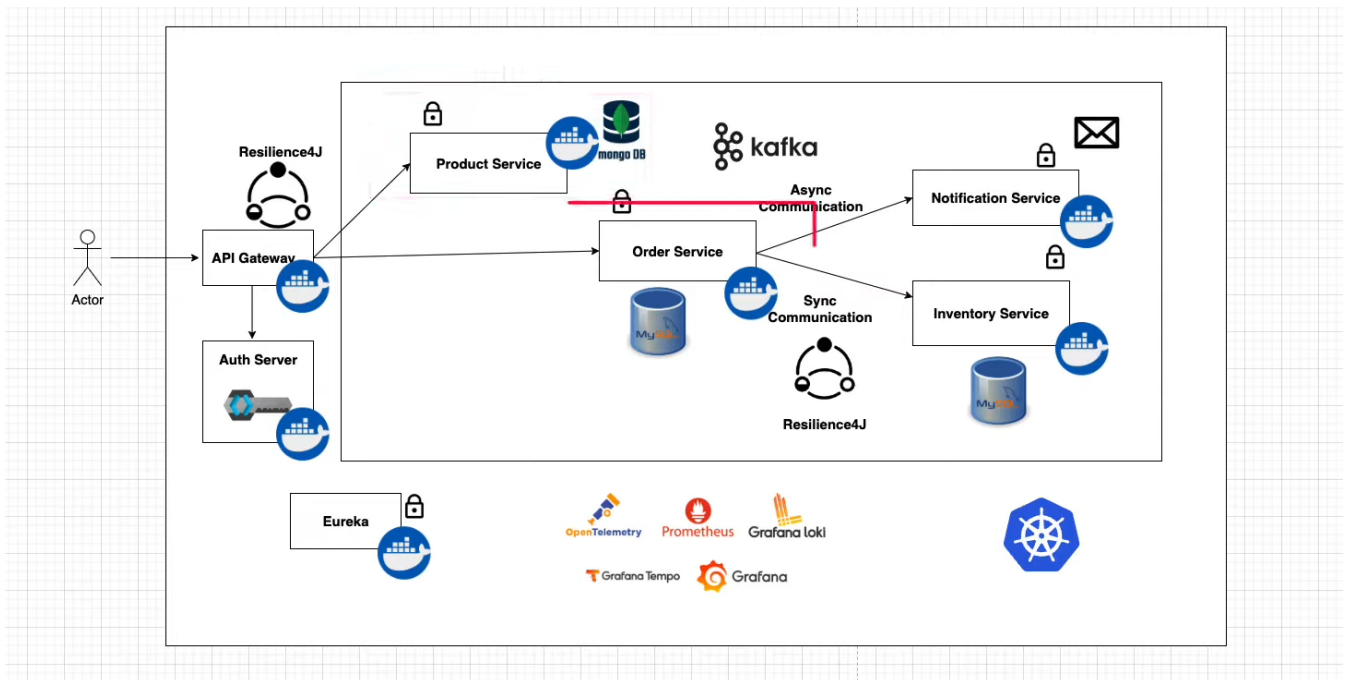
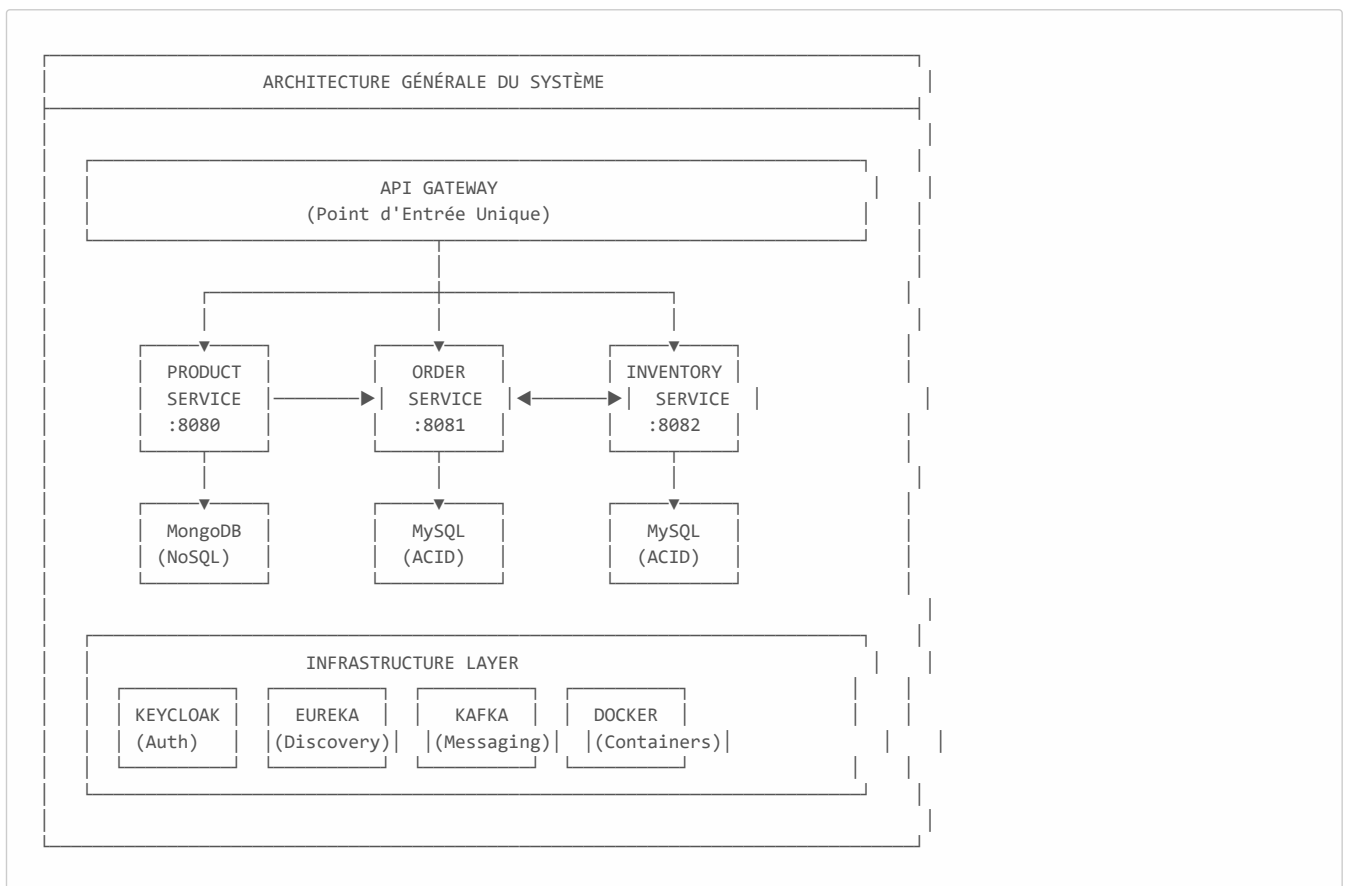


Figure 4.1 : Architecture globale du système montrant les 4 microservices, leurs bases de données respectives, et les composants d'infrastructure



4.1.3 Description Détaillée des Services

📦 Product Service (Port 8080)

Aspect	Détail
Objectif	Gestion du catalogue produits
Base de données	MongoDB (NoSQL)
Justification DB	Schéma flexible pour les attributs produits variables

Fonctionnalités Clés :

- Opérations CRUD complètes sur les produits
- Attributs produits flexibles (sans schéma rigide)
- Performance de lecture optimisée

Order Service (Port 8081)

Aspect	Détail
Objectif	Traitement des commandes clients
Base de données	MySQL (conformité ACID)
Dépendances	Inventory Service via OpenFeign

Fonctionnalités Clés :

- Placement de commandes
- Annulation de commandes
- Validation de l'inventaire avant confirmation
- Gestion sophistiquée des erreurs

Inventory Service (Port 8082)

Aspect	Détail
Objectif	Suivi des niveaux de stock
Base de données	MySQL (intégrité transactionnelle)
Transactions	Supportées pour garantir la cohérence

Fonctionnalités Clés :

- Gestion des stocks (création, mise à jour)
- Opérations de vente (décrémentation)
- Opérations d'achat (incrémentation)
- Validation des niveaux de stock

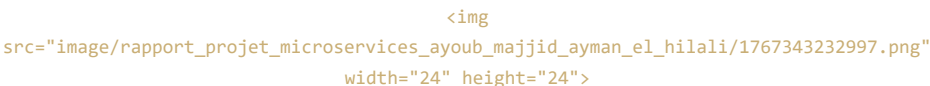
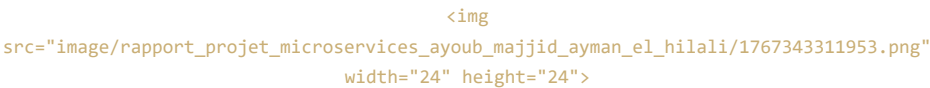
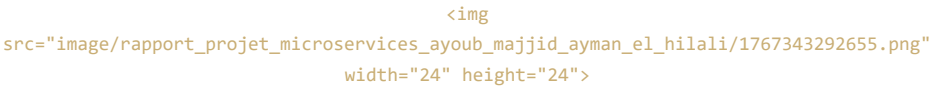
Notification Service (Port 8083) - *Travail Futur*

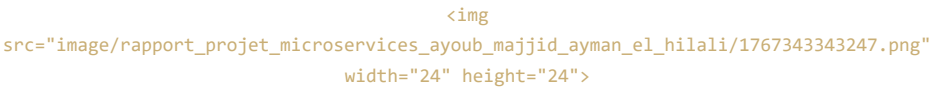
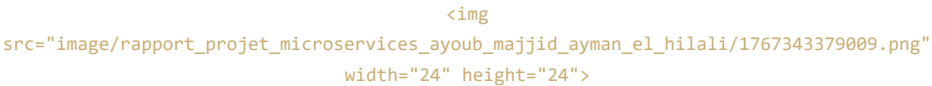
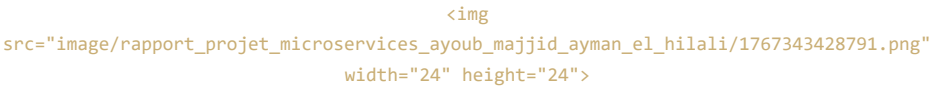
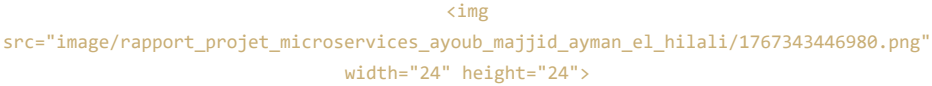
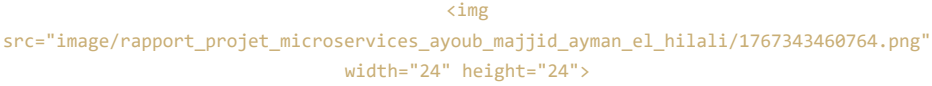
Aspect	Détail
Objectif	Envoi de notifications
Technologie	Consommateur Kafka
Type	Event-driven architecture

Fonctionnalités Clés (Planifiées) :

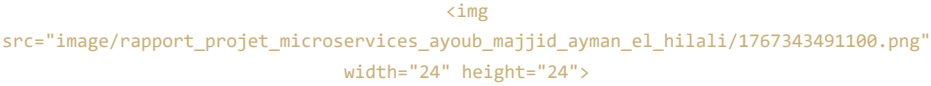
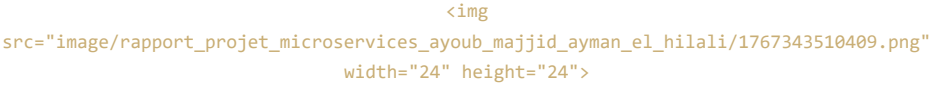
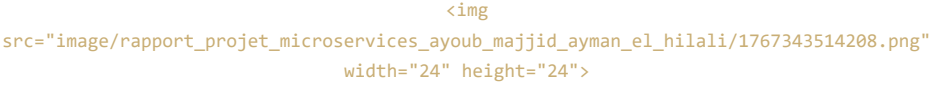
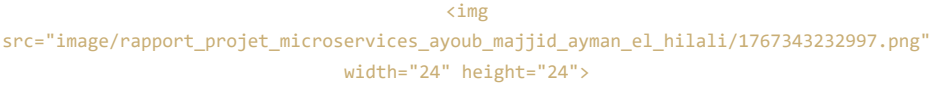
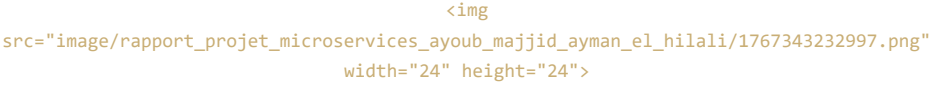
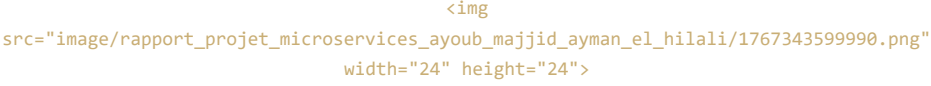
- Traitement asynchrone des messages
- Notifications Email/SMS
- Architecture orientée événements

4.1.4 Composants d'Infrastructure

Composant	Technologie	Icon	Rôle
API Gateway	Spring Cloud Gateway		Routage, sécurité, centralisation des appels
Service Discovery	Eureka		Enregistrement et découverte des services
Message Broker	Apache Kafka		Communication asynchrone

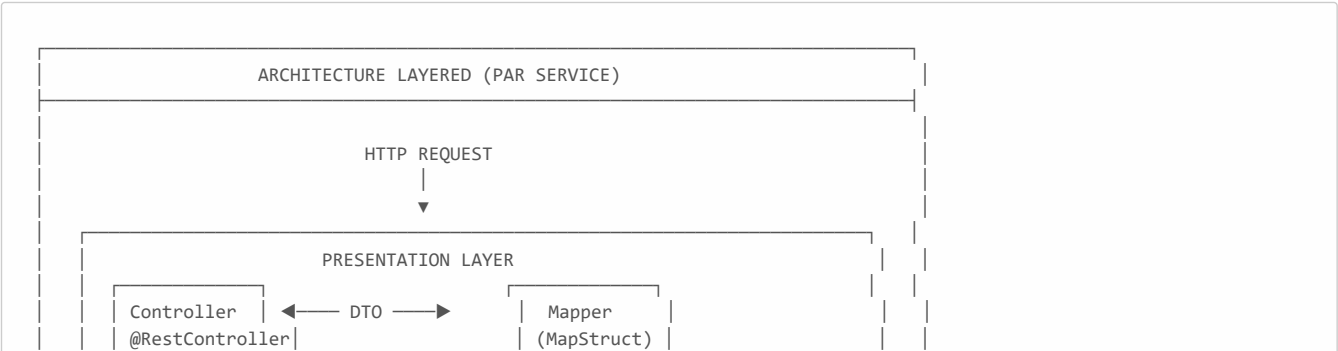
Composant	Technologie	Icon	Rôle
Monitoring	Prometheus + Grafana		Métrique, tableau de bord
Tracing	Zipkin/Tempo		Traçage distribué
Logging	Loki		Journal centralisé
Conteneurisation	Docker		Isolation des services
Orchestration	Kubernetes		Gestion du contenu

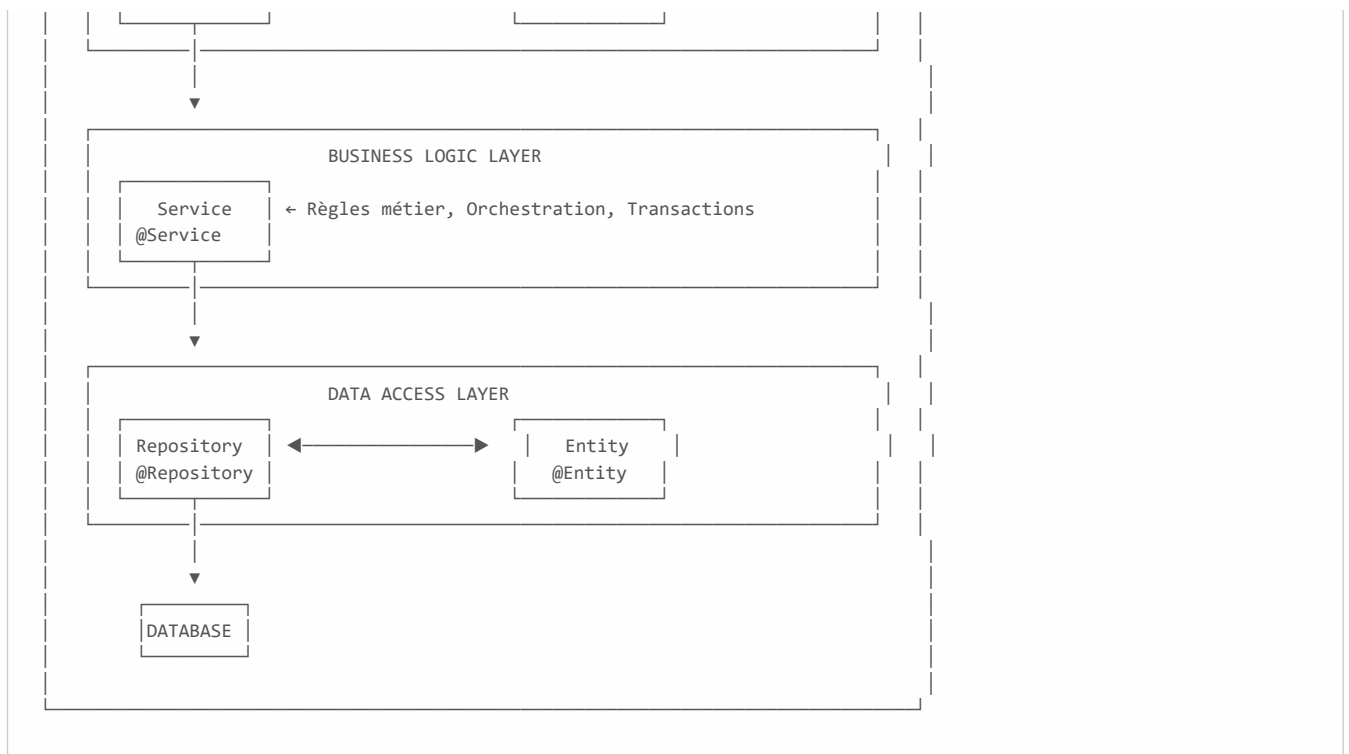
4.1.5 Justification des Choix Technologiques

Composant	Technologie	Icon	Justification
Product Service	MongoDB		Schéma flexible pour les attributs produits variables
Order Service	MySQL		Conformité ACID pour transactions financières
Inventory Service	MySQL		Intégrité transactionnelle pour les mouvements de stock
API Gateway	Spring Cloud Gateway		Routage, sécurité centralisée, filtrage
Communication	OpenFeign		Client HTTP déclaratif, type-safe
Sécurité	Keycloak		IAM open source avec OAuth2/OIDC

4.2 Architecture Interne des Services

Chaque microservice suit une **architecture en couches** standardisée.





Rôles des Couches :

Couche	Responsabilité
Contrôleur	Point d'entrée HTTP, validation, mapping DTO
Service	Logique métier, orchestration, transactions
Repository	Abstraction de l'accès aux données
Entity	Mapping objet-relationnel

4.3 Description des Services

4.3.1 Product Service

Aspect	Détail
Port	8080
Base de données	MongoDB (NoSQL)
Fonction	Gestion du catalogue produits

Endpoints API :

- **POST /products** - Créer un produit
- **GET /products** - Lister les produits
- **GET /products/{id}** - Détails d'un produit
- **PUT /products/{id}** - Modifier un produit
- **DELETE /products/{id}** - Supprimer un produit

4.3.2 Order Service

Aspect	Détail
Port	8081
Base de données	MySQL (ACID)
Fonction	Gestion des commandes
Dépendance	Inventory Service (via OpenFeign)

Endpoints API :

- **POST /orders** - Passer une commande

- `GET /orders` - Lister les commandes
- `GET /orders/{id}` - Détails d'une commande
- `POST /orders/{id}/cancel` - Annuler une commande

4.3.3 Inventory Service

Aspect	Détail
Port	8082
Base de données	MySQL (ACID)
Fonction	Gestion des stocks

Endpoints API :

- `POST /api/inventory` - Créer un stock
- `GET /api/inventory/{sku}` - Vérifier la disponibilité
- `POST /api/inventory/{sku}/sell` - Vendre (décrémenter)
- `POST /api/inventory/{sku}/purchase` - Acheter (incrémenter)

4.4 Flux de Données : Placement d'une Commande



4.5 Schémas de Base de Données

MySQL (Order Service)

```

CREATE TABLE t_orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_number VARCHAR(50) UNIQUE,
  sku_code VARCHAR(100),
  order_status VARCHAR(20),
  price DECIMAL(10,2),
  quantity INT
);

```

```
CREATE TABLE t_inventory (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  sku_code VARCHAR(100) UNIQUE,  
  quantity INT NOT NULL  
);
```

MongoDB (Product Service)

```
{  
  "_id": "ObjectId",  
  "name": "String",  
  "description": "String",  
  "price": "Decimal"  
}
```

4.6 Design Patterns Utilisés

Pattern	Application
Repository	Abstraction de l'accès aux données
DTO	Transfert de données entre couches
Mapper	Conversion Entity ↔ DTO
Service Layer	Encapsulation de la logique métier
API Gateway	Point d'entrée unique

4.7 Conclusion du Chapitre

La conception du système repose sur une architecture microservices bien structurée, avec une séparation claire des responsabilités entre les services et une architecture interne en couches standard.

CHAPITRE 5

RÉALISATION ET IMPLÉMENTATION

5.1 Technologies Utilisées

5.1.1 Stack Technique

Catégorie	Technologie	Version	Rôle
Framework	Spring Boot	4.0.0	Framework applicatif
Cloud	Spring Cloud	2025.1.0	Outils distribués
Langage	Java	21	Langage de programmation
Base NoSQL	MongoDB	Latest	Stockage produits
Base SQL	MySQL	8.x	Stockage commandes/inventaire
Migrations	Flyway	-	Versioning du schéma
Build	Maven	3.9+	Gestion des dépendances
Conteneurs	Docker	Latest	Conteneurisation

5.1.2 Dépendances Clés

```
<!-- Spring Web - API REST -->  
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>  
  
<!-- Spring Data MongoDB -->
```

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

<!-- Spring Cloud OpenFeign -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<!-- Spring Cloud Gateway MVC -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway-mvc</artifactId>
</dependency>

```

5.2 Communication Inter-Services avec OpenFeign

5.2.1 Problème Initial

L'approche traditionnelle avec `RestTemplate` est verbeuse et sujette aux erreurs :

```

// ✗ Ancienne approche avec RestTemplate
RestTemplate restTemplate = new RestTemplate();
String url = "http://localhost:8082/api/inventory/" + skuCode;
ResponseEntity<InventoryResponse> response =
    restTemplate.getForEntity(url, InventoryResponse.class);

```

5.2.2 Solution avec OpenFeign

OpenFeign offre une **approche déclarative** et type-safe :

```

// ☑ Nouvelle approche avec OpenFeign
@FeignClient(name = "inventory-service", url = "${inventory.service.url}")
public interface InventoryClient {

    @GetMapping("/api/inventory/{skuCode}")
    InventoryResponse checkInventory(@PathVariable String skuCode);
}

```

5.2.3 Avantages

Avantage	Description
Déclaratif	Interface simple avec annotations
Type-safe	Vérification à la compilation
Testable	Facilement mockable avec WireMock
Intégré	Support natif de la gestion d'erreurs

5.3 API Gateway

5.3.1 Rôle du Gateway

L'API Gateway centralise les **cross-cutting concerns** :

RÔLES DE L'API GATEWAY		
 SÉCURITÉ • Auth centralisée • Validation JWT	 MONITORING • Logs centralisés • Métriques	 RATE LIMITING • Anti-abus • Quotas
 ROUTAGE • Path-based • Predicates	 LOAD BALANCING • Round-robin • Health checks	 TRANSFORMATION • Header manipulation • Response filtering

5.3.2 Configuration des Routes (Java)

```
@Configuration
public class Routes {

    @Bean
    public RouterFunction<ServerResponse> productServiceRoute() {
        return GatewayRouterFunctions.route("product-service")
            .route(RequestPredicates.path("/api/product/**"),
                request -> {
                    ServerRequest modifiedRequest = ServerRequest.from(request)
                        .uri(URI.create("http://localhost:8080" +
                            request.requestPath().pathWithinApplication()));
                    return HandlerFunctions.http().handle(modifiedRequest);
                })
            .build();
    }
}
```

5.4 Service Discovery avec Eureka

5.4.1 Problématique du Service Discovery

Dans une architecture microservices, les services **scalent dynamiquement**, changeant leurs adresses IP et ports. Coder en dur ces adresses (ex: `localhost:8080`) est **fragile** et **non maintenable**.

Problème	Impact	Solution
Adresses codées en dur	Impossible de scaler	Service Discovery
Instances multiples	Pas de distribution de charge	Load Balancing
Changements d'adresse	Redéploiement requis	Enregistrement dynamique
Défaillances	Pas de failover	Health checks

5.4.2 Architecture Eureka

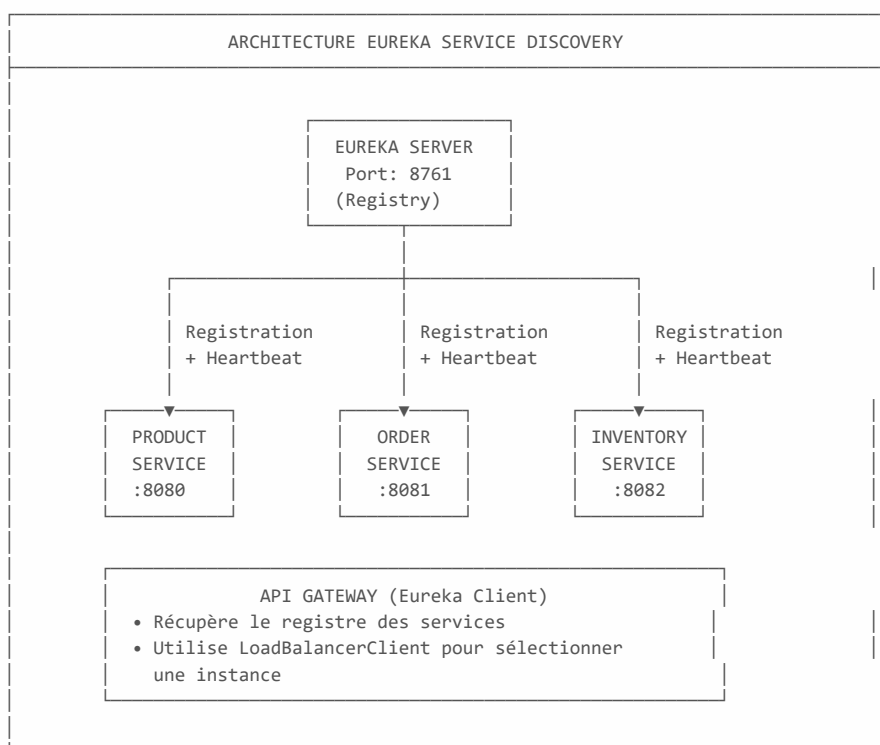


Figure : Espace réservé pour diagramme architecture Eureka

5.4.3 Configuration Eureka Server

```
# Eureka Server Configuration
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServiceApplication.class, args);
    }
}
```

5.4.4 Configuration Eureka Client (Services)

Chaque microservice s'enregistre auprès d'Eureka :

```
# Product Service Configuration
spring.application.name=product-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

5.5 Load Balancing Client-Side

5.5.1 Pourquoi le Load Balancing ?

Eureka répond à la question : **"Quelles instances existent ?"**

Le Load Balancer répond à : **"Quelle instance utiliser MAINTENANT ?"**

EUREKA vs LOAD BALANCER	
EUREKA (Service Registry)	LOAD BALANCER (Instance Selector)
Question: "Qui existe ?"	Question: "Qui choisir ?"
Réponse: - product-service: 192.168.1.5:8080 192.168.1.6:8080 192.168.1.7:8080	Réponse: - Sélectionne 192.168.1.5:8080 - Algorithme: Round-Robin - Prochaine requête: 192.168.1.6:8080
Fréquence: Périodique (30s) Type: Cache local	Fréquence: Par requête Type: Sélection dynamique

5.5.2 Spring Cloud LoadBalancer

Spring Cloud LoadBalancer est une bibliothèque de load balancing **client-side** :

Caractéristique	Valeur
Type	Bibliothèque Java (pas un service)
Localisation	À l'intérieur de l'application Gateway
Communication	Appels de méthodes Java (pas HTTP)

Caractéristique	Valeur
Stratégie par défaut	Round-Robin

5.5.3 Dépendances Maven

```
<!-- Eureka Client -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<!-- Load Balancer -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>
```

5.6 Implémentation du ServiceResolver

5.6.1 Problème avec Gateway MVC

Dans **Gateway MVC**, le schéma `lb://` n'est **pas automatiquement supporté** comme dans Gateway Reactive.

```
// ✗ Ne fonctionne PAS directement dans Gateway MVC
URI.create("lb://product-service/api/product")
```

5.6.2 Solution : ServiceResolver Custom

Nous avons créé un composant `ServiceResolver` qui utilise `LoadBalancerClient` :

```
@Component
public class ServiceResolver {

    private final LoadBalancerClient loadBalancerClient;

    public ServiceResolver(LoadBalancerClient loadBalancerClient) {
        this.loadBalancerClient = loadBalancerClient;
    }

    /**
     * Résout un nom de service en URI physique via Eureka.
     *
     * @param serviceName Nom logique (ex: "product-service")
     * @param path Chemin à ajouter (ex: "/api/product")
     * @return URI résolu (ex: "http://192.168.1.5:8080/api/product")
     */
    public URI resolve(String serviceName, String path) {
        // 1. Demander au LoadBalancer de choisir une instance
        ServiceInstance instance = loadBalancerClient.choose(serviceName);

        if (instance == null) {
            throw new IllegalStateException(
                "No instance available for service: " + serviceName
            );
        }

        // 2. Construire l'URI complet
        return URI.create(instance.getUri().toString() + path);
    }
}
```

5.6.3 Flux de Résolution

FLUX DE RÉOLUTION DE SERVICE

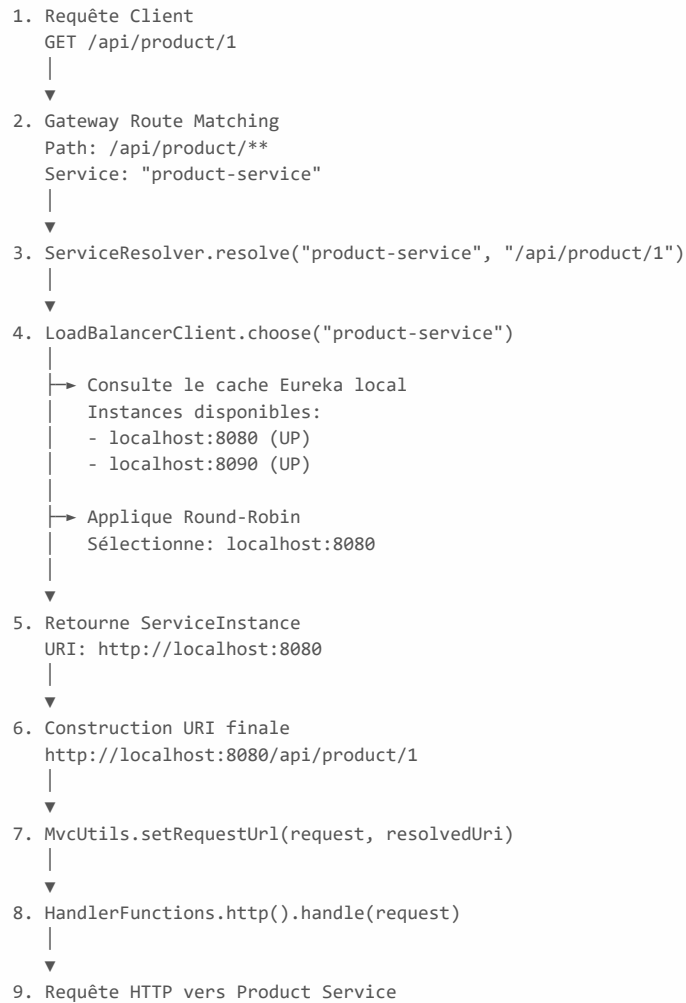


Figure : Espace réservé pour diagramme flux de résolution

5.7 Intégration dans Routes.java

5.7.1 Configuration des Routes avec Eureka

```

@Configuration
public class Routes {

    private static final String PRODUCT_SERVICE = "product-service";
    private static final String ORDER_SERVICE = "order-service";
    private static final String INVENTORY_SERVICE = "inventory-service";

    private final ServiceResolver serviceResolver;

    public Routes(ServiceResolver serviceResolver) {
        this.serviceResolver = serviceResolver;
    }

    /**
     * Méthode générique pour créer une route de service.
     */
    private RouterFunction<ServerResponse> createServiceRoute(
        String serviceName,
        String pathPrefix
    ) {
        return GatewayRouterFunctions.route(serviceName)
            .route(RequestPredicates.path(pathPrefix), request -> {
                // 1. Extraire le chemin de la requête
                String path = request.requestPath()
            })
    }
}
  
```

```

        .pathWithinApplication()
        .value();

// 2. Résoudre le service via Eureka
URI resolvedUri = serviceResolver.resolve(serviceName, path);

log.debug(">>> Routing {} to {}: {}",
    request.method(), serviceName, resolvedUri);

// 3. Transférer le header Authorization (JWT)
String authHeader = request.headers()
    .firstHeader("Authorization");

ServerRequest newRequest = ServerRequest.from(request)
    .header("Authorization", authHeader)
    .build();

// 4. Utiliser MvcUtils pour définir l'URL résolue
MvcUtils.setRequestUrl(newRequest, resolvedUri);

// 5. Forwarder la requête
return HandlerFunctions.http().handle(newRequest);
})
.filter(CircuitBreakerFilterFunctions.circuitBreaker(
    serviceName + "CircuitBreaker",
    URI.create("forward:/fallbackRoute")
))
.build();
}

@Bean
public RouterFunction<ServerResponse> productServiceRoute() {
    return createServiceRoute("product-service", "/api/product/**");
}

@Bean
public RouterFunction<ServerResponse> orderServiceRoute() {
    return createServiceRoute("order-service", "/api/order/**");
}

@Bean
public RouterFunction<ServerResponse> inventoryServiceRoute() {
    return createServiceRoute("inventory-service", "/api/inventory/**");
}
}

```

5.7.2 Explication de MvcUtils.setRequestUrl()

`MvcUtils.setRequestUrl()` est une **classe utilitaire** de Spring Cloud Gateway MVC qui :

Fonction	Description
Objectif	Modifier l'URL de destination d'une requête
Usage	Remplacer l'URL logique par l'URL physique résolue
Effet	La requête HTTP sera envoyée à l'URI fourni

```

// AVANT MvcUtils.setRequestUrl()
// request.uri() = /api/product/1 (URL logique)

MvcUtils.setRequestUrl(newRequest, URI.create("http://localhost:8080/api/product/1"));

// APRÈS MvcUtils.setRequestUrl()
// La requête sera envoyée à http://localhost:8080/api/product/1

```

5.7.3 Pourquoi cette Synchronisation est Nécessaire

AVANTAGES DE LA SYNCHRONISATION EUREKA

- ☒ **Scaling Dynamique**
 - Ajout d'instances sans redémarrage du Gateway
 - Product Service: 1 instance → 5 instances
 - Gateway découvre automatiquement les nouvelles instances
- ☒ **Load Balancing Automatique**
 - Distribution Round-Robin entre instances
 - Pas de configuration manuelle
- ☒ **Failover**
 - Si une instance tombe, Eureka la marque DOWN
 - LoadBalancer ne la sélectionne plus
 - Trafic redirigé vers instances saines
- ☒ **Zero Downtime Deployment**
 - Déploiement progressif (Blue-Green)
 - Anciennes instances restent actives pendant le déploiement

5.8 Gateway MVC vs Gateway Reactive

5.8.1 Comparaison Technique

Aspect	Gateway MVC (Notre Choix)	Gateway Reactive
Stack	Servlet API (Tomcat)	WebFlux (Netty)
Modèle	Blocking / Thread-per-request	Non-blocking / Event Loop
Load Balancing	Nécessite <code>ServiceResolver</code> custom	Support natif <code>lb://</code>
Debugging	Stack traces standard (facile)	Stack traces async (difficile)
Compatibilité	Services JDBC/JPA	Services réactifs
Complexité	Plus simple	Plus complexe

5.8.2 Pourquoi Gateway MVC ?

Notre choix de **Gateway MVC** est justifié par :

1. **Cohérence** : Nos microservices utilisent Spring MVC (blocking)
2. **SimPLICITÉ** : Équipe familière avec le modèle impératif
3. **Debugging** : Stack traces plus claires
4. **Intégration** : Compatibilité avec JDBC, JPA

5.8.3 Schéma `lb://` dans Gateway Reactive

Dans Gateway Reactive, le support est natif :

```
# Gateway Reactive - Configuration YAML
spring:
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://product-service # ☒ Fonctionne directement
          predicates:
            - Path=/api/product/**
```

Dans Gateway MVC, nous devons **manuellement** résoudre via `ServiceResolver`.

5.9 Flux Complet de Requête

```
FLUX COMPLET CLIENT → SERVICE

1. Client
```

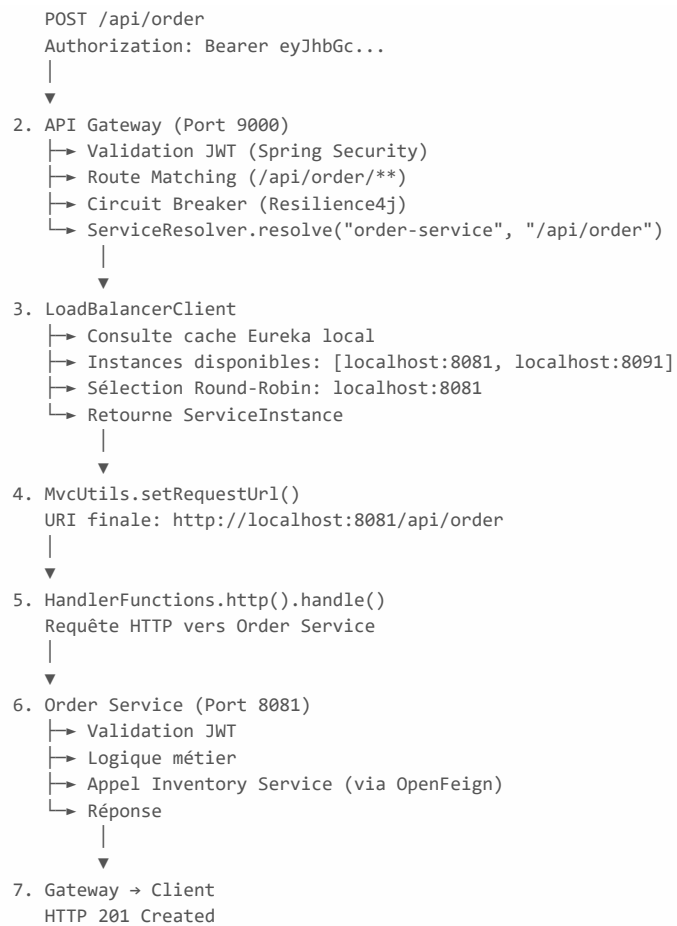
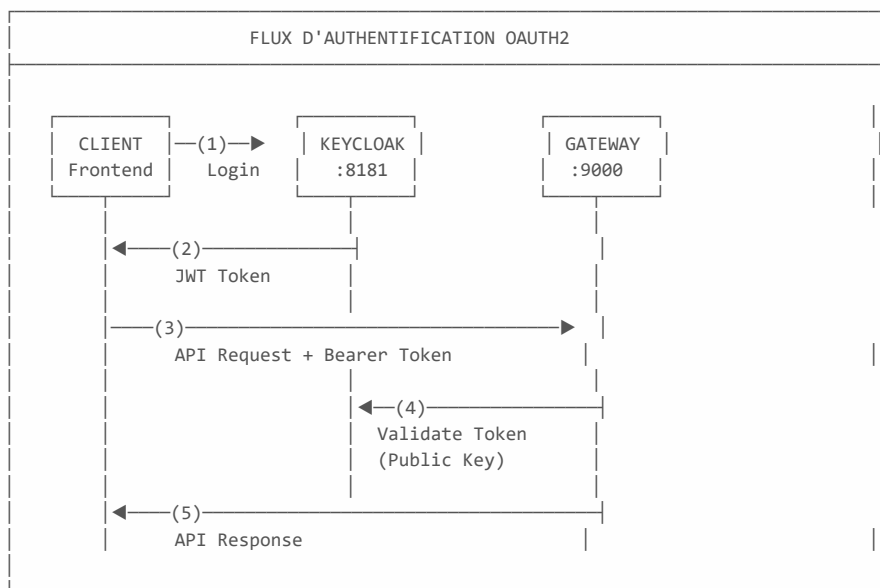


Figure : Espace réservé pour diagramme flux complet

5.10 Sécurité avec Keycloak

5.10.1 Architecture de Sécurité



5.10.2 Configuration Spring Security

```

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(authorize -> authorize
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            )
            .build();
    }
}

```

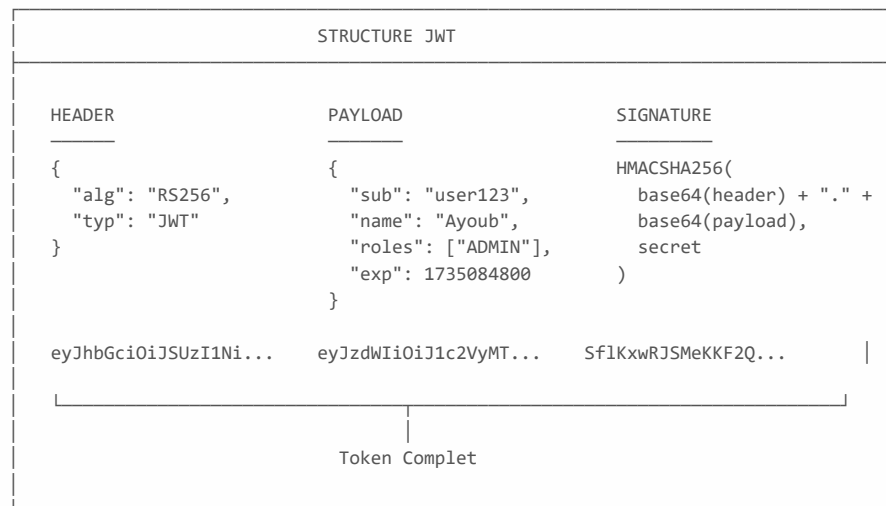
5.10.3 Configuration application.properties

```

spring.security.oauth2.resourceserver.jwt.issuer-uri=
http://localhost:8181/realms/spring-microservices-security-realm

```

5.10.4 Structure du Token JWT



5.10.5 Types de Tokens Keycloak

Access Token vs Refresh Token

Caractéristique	Access Token	Refresh Token
Objectif	Autoriser les requêtes API	Obtenir de nouveaux access tokens
Durée de vie	Courte (5 min)	Longue (30 min - 24h)
Envoyé à	Gateway / APIs	Uniquement à Keycloak
Contenu	Infos utilisateur, rôles	Juste un identifiant de référence
Si volé	Dommages limités (expire vite)	Peut obtenir de nouveaux tokens

Cycle de Vie des Tokens



Obtention refresh_token (expire dans 30 min)

0:30 → L'utilisateur clique "Charger Produits"
↳ Token valide ☒ → Appel API

5:30 → L'utilisateur clique "Ajouter au Panier"
↳ Token expiré ☒ → Utiliser refresh_token
↳ Obtenir nouveau access_token → Appel API ☒

35:00 → L'utilisateur revient après une pause
↳ Token expiré ☒ → Refresh aussi expiré ☒
↳ Redirection vers la page de connexion

5.10.6 Types de Clients Keycloak

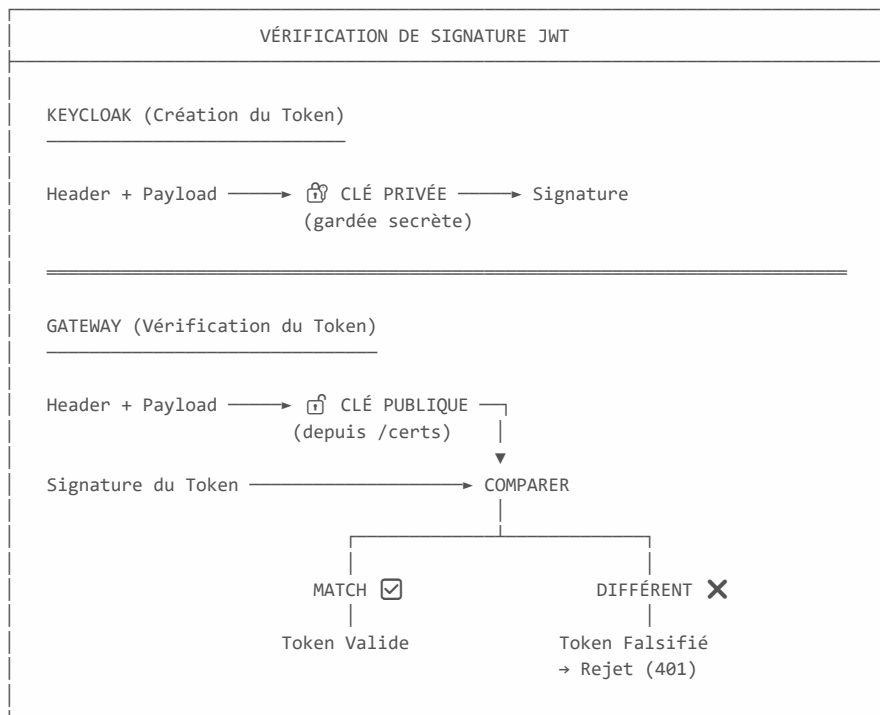
Type	Cas d'Usage	Secret ?	Exemple
Public	Apps navigateur (SPA), Mobile	☒ Non	React, Vue, Angular
Confidential	Apps serveur	☒ Oui	Backend Spring Boot
Bearer-only	Services API	N/A	Microservices qui valident uniquement les tokens

Notre Configuration

Pour notre projet, nous utilisons un **client public** (**frontend-app**) pour la page de login personnalisée :

Paramètre	Valeur	Justification
Client Authentication	OFF	Client public (pas de secret)
Direct Access Grants	ON	Permet le Password Grant Flow
Standard Flow	OFF	Pas de callback redirect

5.10.7 Vérification de la Signature JWT



5.10.8 Contrôle d'Accès Basé sur les Rôles (RBAC)

Endpoint	Rôle Requis
----------	-------------

Endpoint	Rôle Requis
GET /api/products	USER, ADMIN
POST /api/products	ADMIN
DELETE /api/products/{id}	ADMIN
POST /orders	USER, ADMIN

5.11 Conteneurisation avec Docker

5.11.1 Docker Compose - Product Service

```

services:
  mongodb:
    image: mongo:latest
    container_name: mongodb
    ports:
      - "27017:27017"
    environment:
      - MONGO_INITDB_ROOT_USERNAME=root
      - MONGO_INITDB_ROOT_PASSWORD=supersecretpassword
      - MONGO_INITDB_DATABASE=product-service
    volumes:
      - mongodb_data:/data/db

volumes:
  mongodb_data:

```

5.11.2 Docker Compose - Order Service (MySQL)

```

services:
  mysql:
    image: mysql:8.3.0
    container_name: mysql_order_service
    environment:
      MYSQL_ROOT_PASSWORD: mysql
      MYSQL_DATABASE: order_service
    ports:
      - "3306:3306"
    volumes:
      - ./mysql_data:/var/lib/mysql

```

5.12 Résilience avec Resilience4j

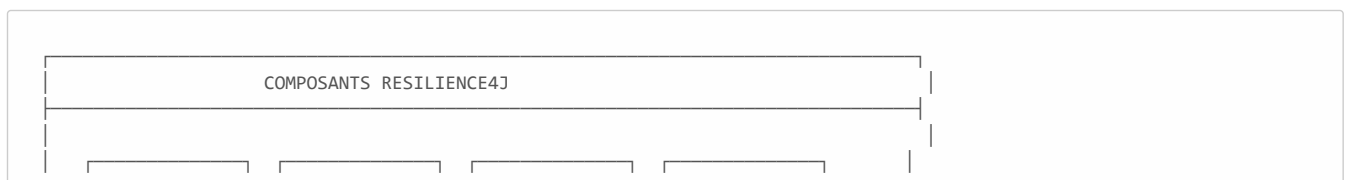
5.12.1 Problématique de la Résilience

Dans une architecture microservices, la **défaillance d'un service** peut entraîner des **effets en cascade** sur l'ensemble du système. Les problèmes courants incluent :

Problème	Impact	Solution
Service lent	Threads bloqués, timeout	TimeLimiter
Service défaillant	Erreurs répétées	Circuit Breaker
Pic de trafic	Surcharge système	Rate Limiter
Erreurs transitoires	Échecs temporaires	Retry

5.12.2 Spring Cloud Circuit Breaker avec Resilience4j

Resilience4j est une bibliothèque légère de tolérance aux pannes inspirée de Netflix Hystrix.



CIRCUIT BREAKER	TIME LIMITER	RATE LIMITER	RETRY
Protège contre les défaillances	Limite temps d'attente	- Contrôle trafic - Anti-abus	Réessaie appels échoués

5.12.3 Dépendances Maven

```

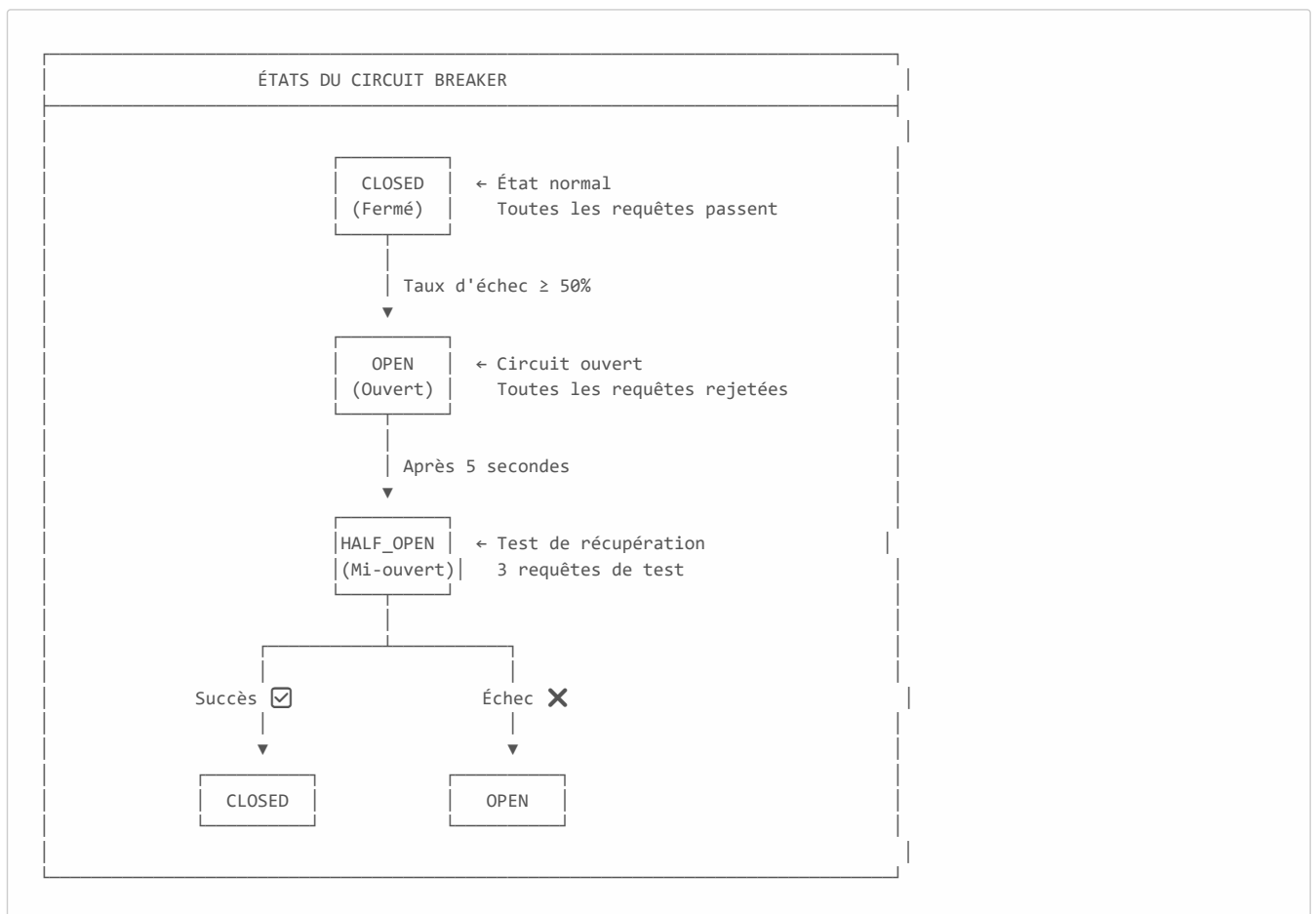
<!-- Circuit Breaker avec Resilience4j -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-reactor-resilience4j</artifactId>
</dependency>

<!-- Actuator pour monitoring -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

```

5.13 Circuit Breaker Pattern

5.13.1 Machine à États du Circuit Breaker



5.13.2 Configuration du Circuit Breaker

```

# Actuator Endpoints
management.health.circuitbreakers.enabled=true
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

```

```
# Circuit Breaker Configuration
resilience4j.circuitbreaker.configs.default.register-health-indicator=true
resilience4j.circuitbreaker.configs.default.sliding-window-type=COUNT_BASED
resilience4j.circuitbreaker.configs.default.sliding-window-size=10
resilience4j.circuitbreaker.configs.default.minimum-number-of-calls=5
resilience4j.circuitbreaker.configs.default.failure-rate-threshold=50
resilience4j.circuitbreaker.configs.default.wait-duration-in-open-state=5s
resilience4j.circuitbreaker.configs.default.permitted-number-of-calls-in-half-open-state=3
resilience4j.circuitbreaker.configs.default.automatic-transition-from-open-to-half-open-enabled=true
```

5.13.3 Explication des Propriétés

Propriété	Valeur	Signification
sliding-window-type	COUNT_BASED	Mesure basée sur les N derniers appels
sliding-window-size	10	Nombre d'appels à observer
minimum-number-of-calls	5	Minimum d'appels avant évaluation
failure-rate-threshold	50%	Seuil d'ouverture du circuit
wait-duration-in-open-state	5s	Durée en état OPEN
permitted-calls-in-half-open	3	Appels de test en HALF_OPEN

5.13.4 Exemple de Scénario

Historique des 10 derniers appels :

✓ ✓ ✗ ✗ ✗ ✗ ✗ ✓ ✓ ✗

Calcul :

- Total appels : 10
- Échecs : 6
- Taux d'échec : 60%

Décision :

60% ≥ 50% → Circuit passe à OPEN

Comportement :

- T0 → Circuit OPEN
- T0-T5s → Toutes les requêtes rejetées immédiatement
- T5s → Transition automatique vers HALF_OPEN
 - 3 requêtes de test autorisées
 - Si succès → CLOSED
 - Si échec → OPEN

Figure : Espace réservé pour diagramme du scénario Circuit Breaker

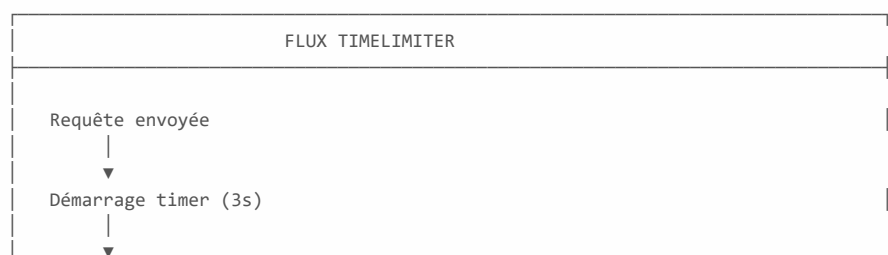
5.14 TimeLimiter Pattern

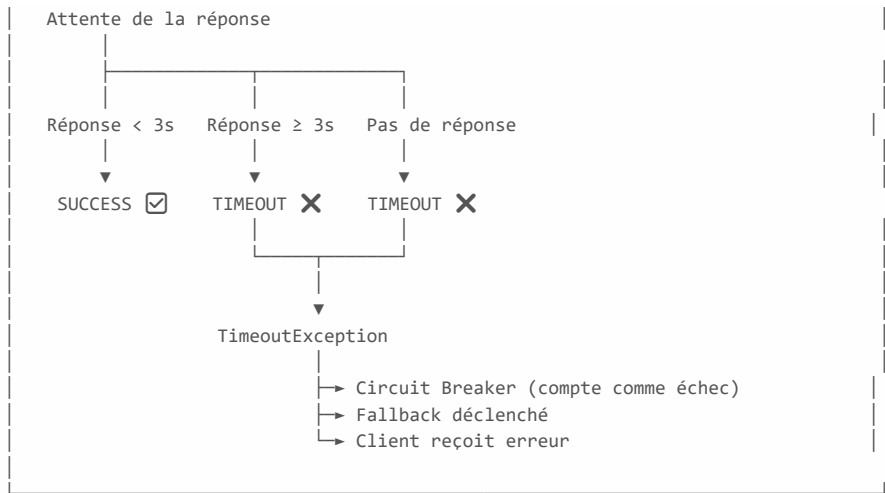
5.14.1 Rôle du TimeLimiter

Le **TimeLimiter** protège contre les **services lents** en limitant le temps d'attente maximum.

```
resilience4j.timelimiter.configs.default.timeout-duration=3s
```

5.14.2 Flux d'Exécution





5.14.3 Exemple Pratique

Scénario : Le Product Service est lent à cause d'un verrou de base de données.

Service répond en 6 secondes
 → `Timelimiter` arrête à 3s
 → `TimeoutException` lancée
 → Circuit Breaker enregistre un ÉCHEC
 → Après 5 échecs → Circuit OPEN
 → Gateway retourne fallback instantanément
 → Système reste réactif

Figure : Espace réservé pour timeline `Timelimiter`

5.15 Rate Limiter Pattern

5.15.1 Configuration

```
resilience4j.ratelimiter.configs.default.timeout-duration=3s
```

5.15.2 Différence avec Circuit Breaker

Aspect	Circuit Breaker	Rate Limiter
Protection contre	Défaillances	Pics de trafic
Logique	Basée sur échecs	Basée sur temps
État HALF_OPEN	✓ Oui	✗ Non
Limite par seconde	✗ Non	✓ Oui

5.16 Retry Pattern

5.16.1 Configuration

```
resilience4j.retry.configs.default.max-attempts=3
resilience4j.retry.configs.default.wait-duration=2s
```

5.16.2 Logique de Retry

Tentative #1 → Échec
 ↓ (attente 2s)
 Tentative #2 → Échec

↓ (attente 2s)
Tentative #3 → Succès ☒

5.16.3 Interaction avec Circuit Breaker

⚠ **Attention** : Les retries comptent comme des appels individuels pour le Circuit Breaker.

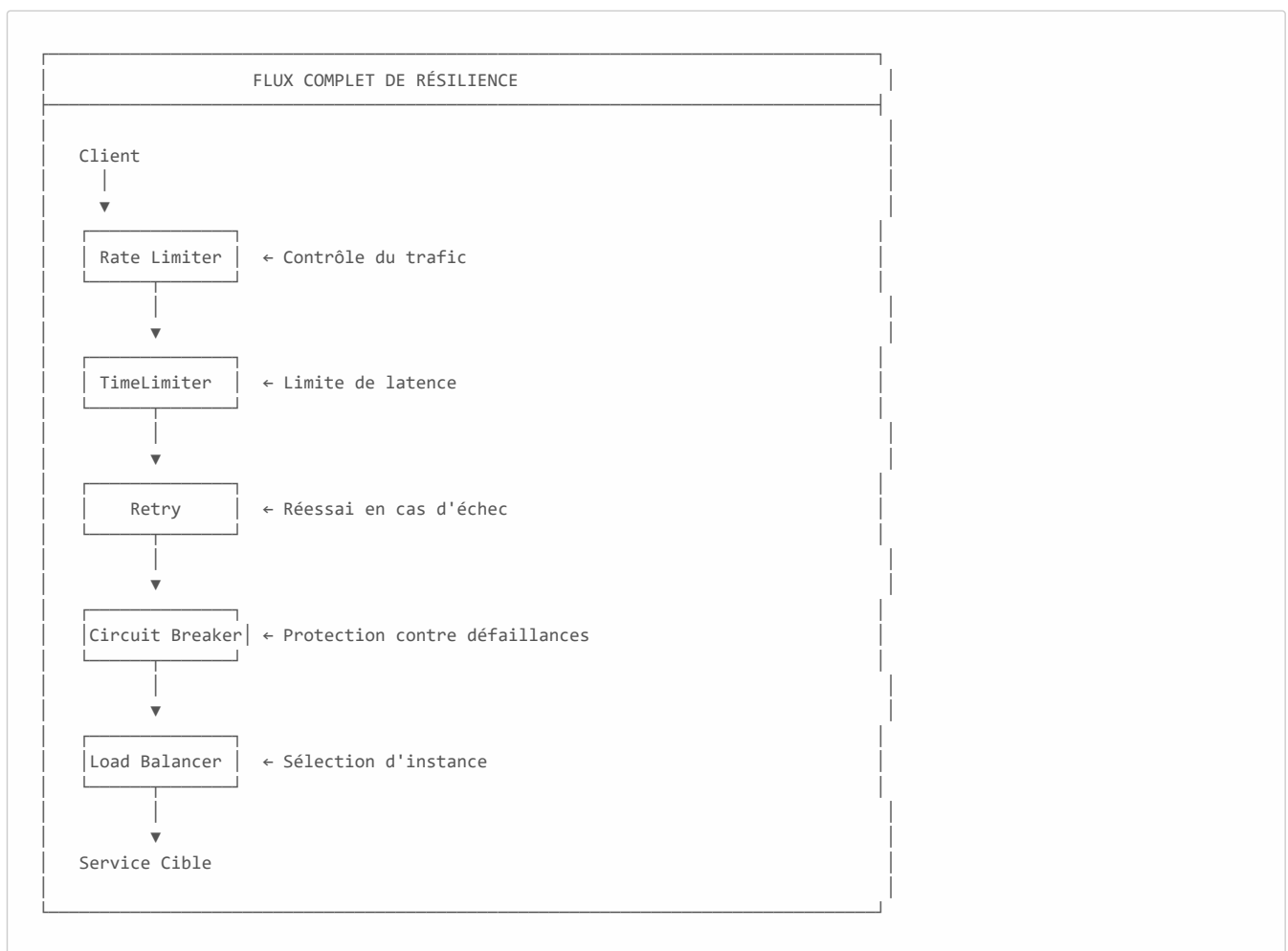
1 requête utilisateur avec 3 tentatives
→ 3 appels enregistrés par le Circuit Breaker

2 requêtes utilisateur × 3 tentatives = 6 appels
→ Peut ouvrir le circuit rapidement

5.16.4 Bonnes Pratiques

Cas d'Usage	Retry ?
Erreurs réseau transitoires	☒ Oui
Service en démarrage	☒ Oui
Timeouts occasionnels	☒ Oui
Service surchargé	☒ Non
Erreurs de validation (4xx)	☒ Non

5.17 Architecture Complète de Résilience



5.17.1 Monitoring via Actuator

L'état du Circuit Breaker est exposé via Spring Boot Actuator :

```

{
  "status": "DOWN",
  "components": {
    "circuitBreakers": {
      "status": "DOWN",
      "details": {
        "inventoryService": {
          "state": "OPEN",
          "failureRate": "60%",
          "slowCallRate": "0%",
          "bufferedCalls": 10,
          "failedCalls": 6
        }
      }
    }
  }
}

```

Endpoint : GET <http://localhost:8081/actuator/health>

Figure : Espace réservé pour capture d'écran Actuator

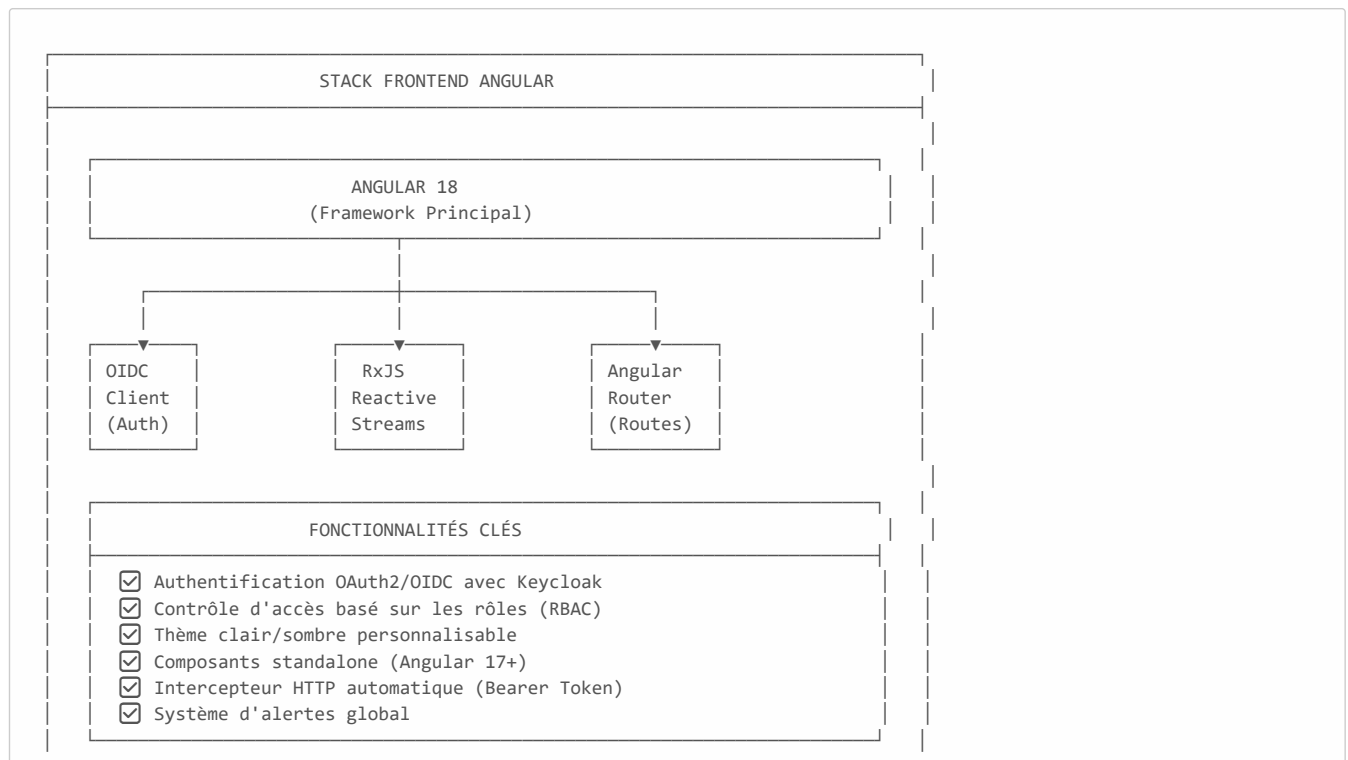
5.18 Modèle Mental des Patterns de Résilience

Pattern	Question Clé
TimeLimiter	"Combien de temps puis-je attendre ?"
RateLimiter	"Puis-je envoyer cette requête maintenant ?"
CircuitBreaker	"Dois-je même essayer ?"
Retry	"Et si je réessayais ?"
LoadBalancer	"Vers quelle instance envoyer ?"
Eureka	"Quelles instances existent ?"

5.19 Frontend Angular - Démonstration

5.19.1 Stack Technologique Frontend

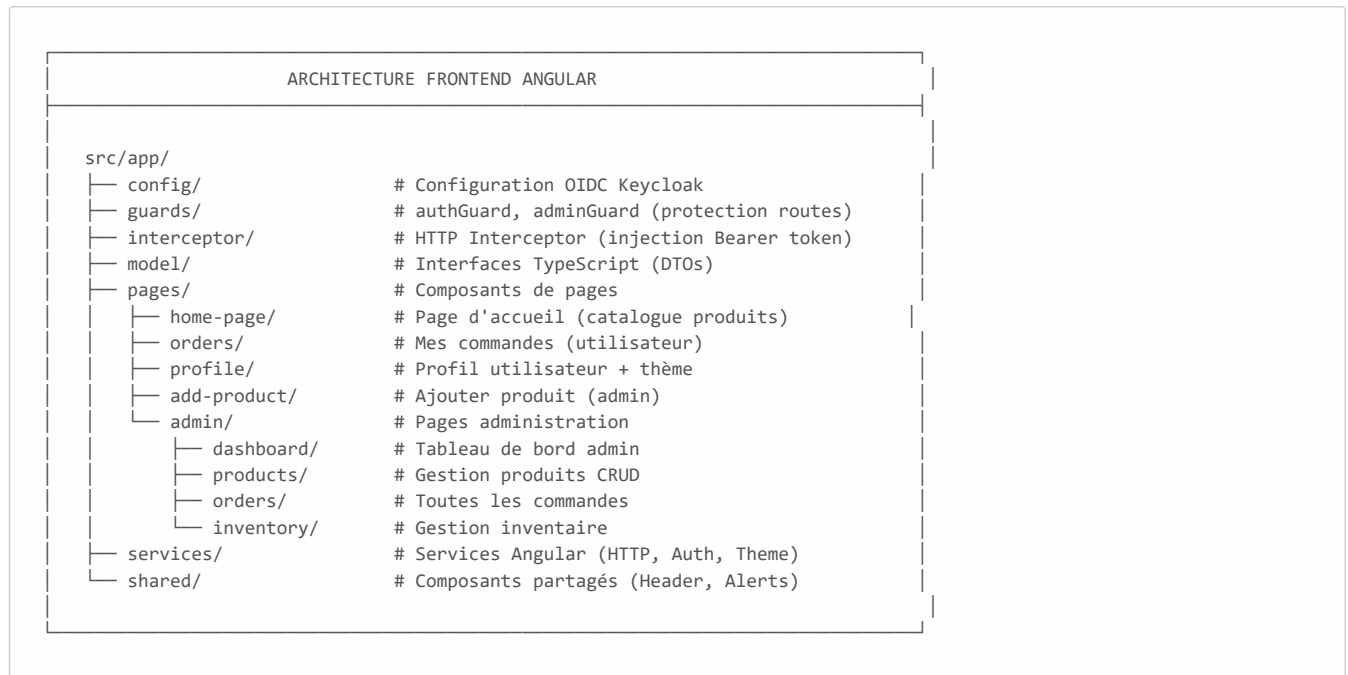
Notre application frontend est développée avec **Angular 18**, un framework moderne et robuste pour la création d'applications web SPA (Single Page Application).



5.19.2 Technologies Utilisées

Technologie	Icon	Version	Rôle
Angular	<!-- ICON: Angular -->	18.0.0	Framework SPA
TypeScript	<!-- ICON: TypeScript -->	5.4	Langage typé
RxJS	<!-- ICON: RxJS -->	7.8	Programmation réactive
angular-auth-oidc-client	<!-- ICON: OpenID -->	17.1.0	Authentification OIDC
CSS3	<!-- ICON: CSS3 -->	-	Thème moderne avec variables CSS

5.19.3 Architecture Frontend

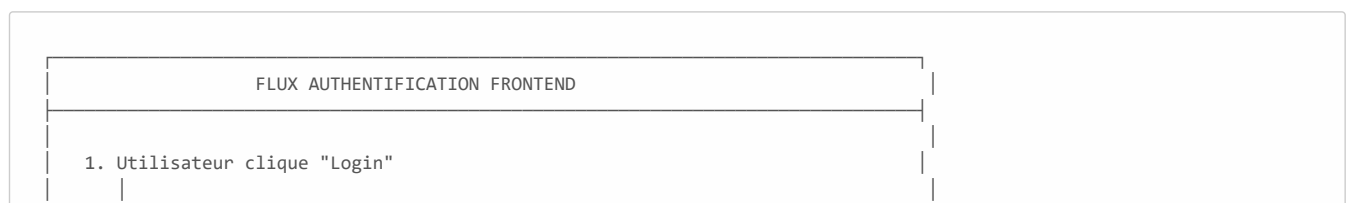


5.19.4 Fonctionnalités Implémentées

Gestion des Rôles (RBAC)

Fonctionnalité	Client (USER)	Admin (ADMIN)	Invité
Voir les produits	☒	☒	☒
Passer une commande	☒	☒	☒
Voir ses commandes	☒	☒	☒
Annuler une commande	☒	☒	☒
Gérer les produits (CRUD)	☒	☒	☒
Voir toutes les commandes	☒	☒	☒
Gérer l'inventaire	☒	☒	☒
Accéder au profil	☒	☒	☒

Flux d'Authentification





5.19.5 Démonstration des Pages

5.19.5.0 Pages d'Authentification Keycloak

Technologie : Keycloak (IAM Open-Source)

URL : `http://localhost:9001/realms/spring-microservices-security-realm`

Description :

L'authentification est gérée par **Keycloak**, une solution IAM (Identity and Access Management) open-source. Lorsqu'un utilisateur clique sur "Login", il est redirigé vers les pages Keycloak pour s'authentifier ou créer un compte.

Fonctionnalités :

- Page de connexion avec thème personnalisé
- Création de compte utilisateur
- Récupération de mot de passe
- Single Sign-On (SSO)
- Support multi-facteurs (optionnel)

Figure 5.19.0a — Page de connexion Keycloak

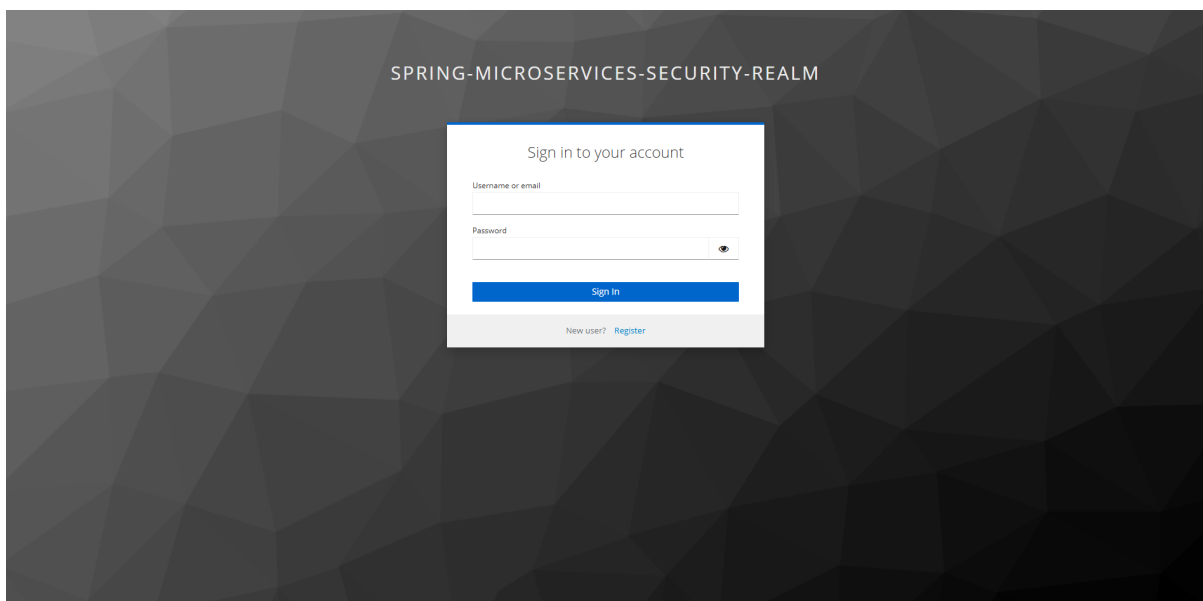


Figure 5.19.0b — Page d'inscription Keycloak

The image shows the Keycloak registration page for the realm 'SPRING-MICROSERVICES-SECURITY-REALM'. The page has a dark, geometric background. A white registration form is centered, titled 'Register'. It includes fields for Username, Password, Confirm password, Email, First name, and Last name, all marked as required. There are toggle icons for password visibility. At the bottom of the form, there is a link to 'Back to Login' and a blue 'Register' button.

5.19.5.1 Page d'Accueil - Catalogue Produits

Route : /

Accès : Public

Description :

La page d'accueil présente le catalogue des produits disponibles avec une interface moderne. Les utilisateurs peuvent :

- Rechercher des produits par nom ou ID
- Voir les détails et prix des produits
- Commander des produits (si authentifié en tant que CLIENT)

Fonctionnalités :

- Barre de recherche dynamique
- Grille de produits responsive
- Bouton "Order Now" avec quantité (pour clients)
- Badge "Admin View" (pour administrateurs)

Figure 5.19.1 — Page d'accueil avec catalogue produits (Mode Sombre)

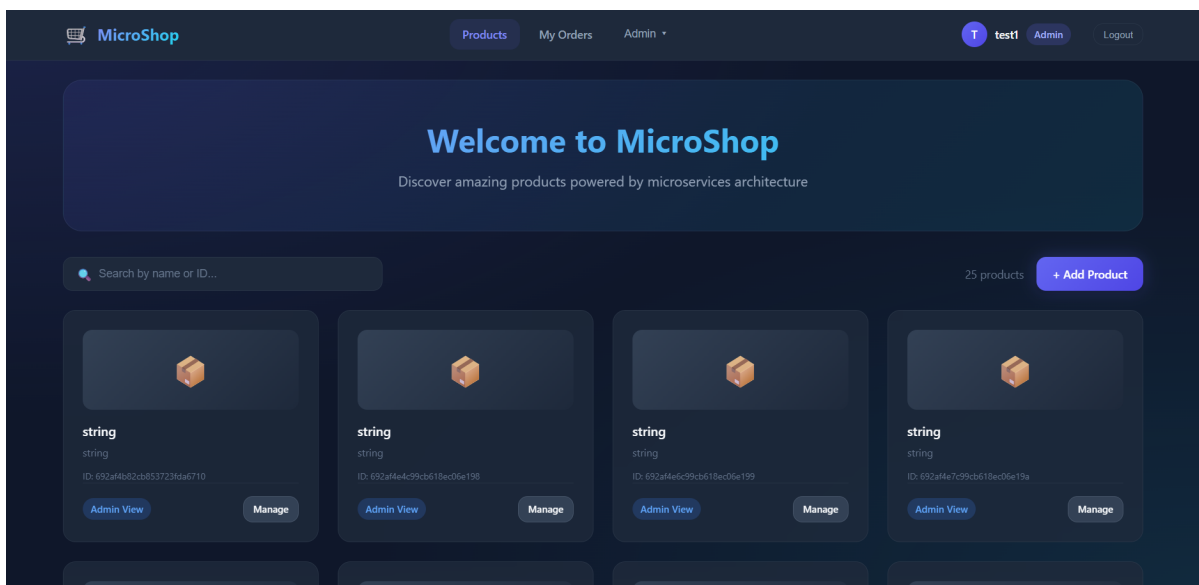
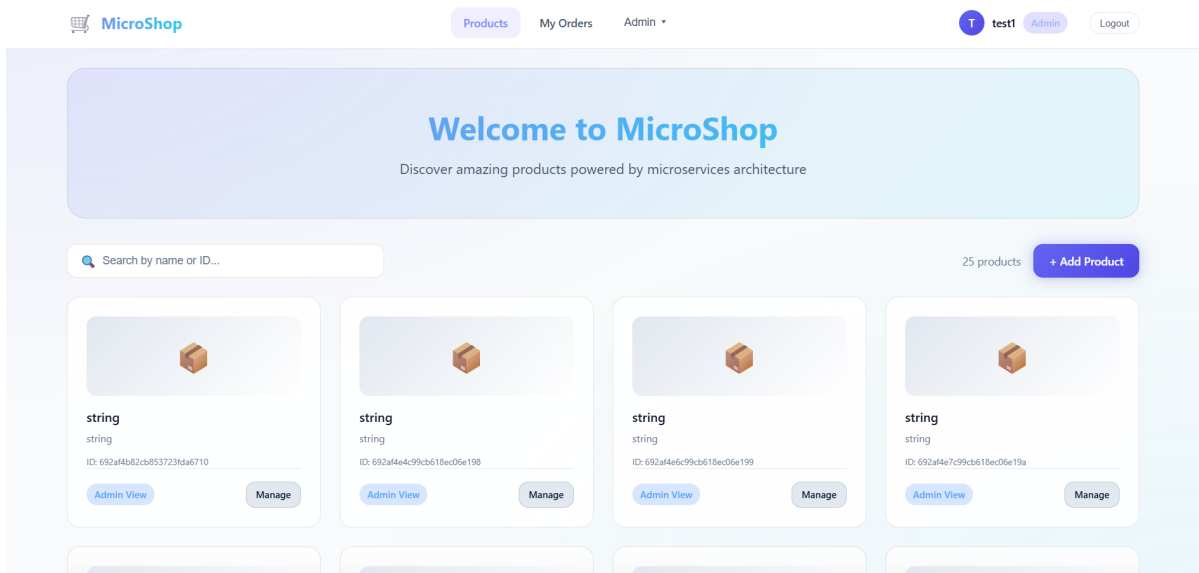


Figure 5.19.2 — Page d'accueil en mode clair



5.19.5.2 Page Mes Commandes

Route : `/orders`

Accès : Utilisateurs authentifiés

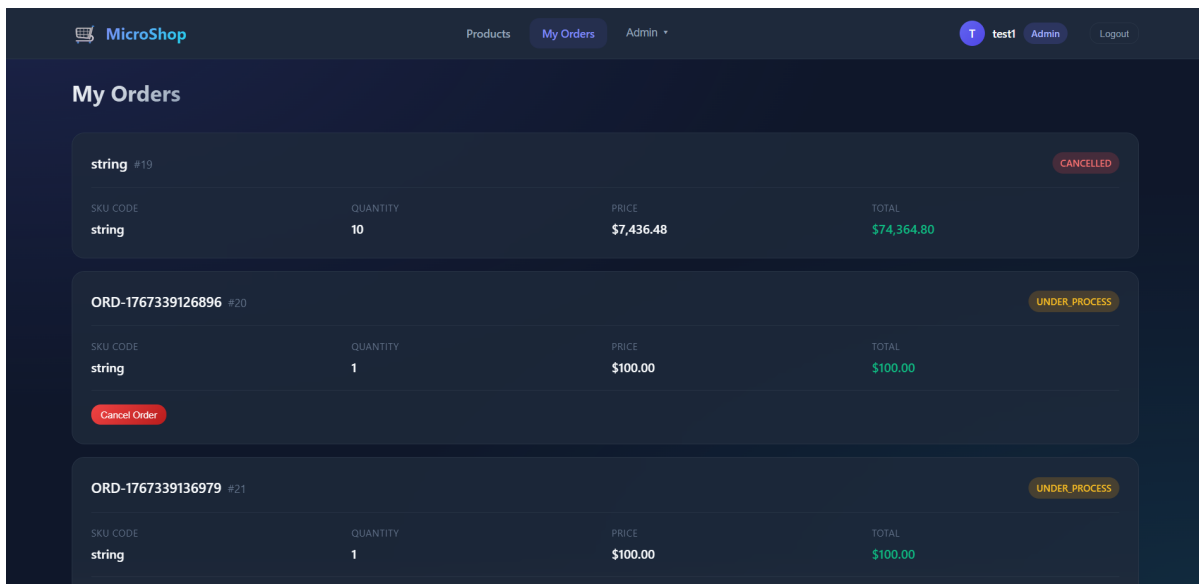
Description :

Cette page permet aux utilisateurs de consulter l'historique de leurs commandes et d'annuler les commandes en cours de traitement.

Fonctionnalités :

- Liste des commandes avec statut (badge coloré)
- Détails : numéro de commande, SKU, quantité, prix, total
- Bouton "Cancel Order" pour les commandes UNDER_PROCESS
- Message d'état vide si aucune commande

Figure 5.19.3 — Page Mes Commandes



5.19.5.3 Page Profil Utilisateur

Route : `/profile`

Accès : Utilisateurs authentifiés

Description :

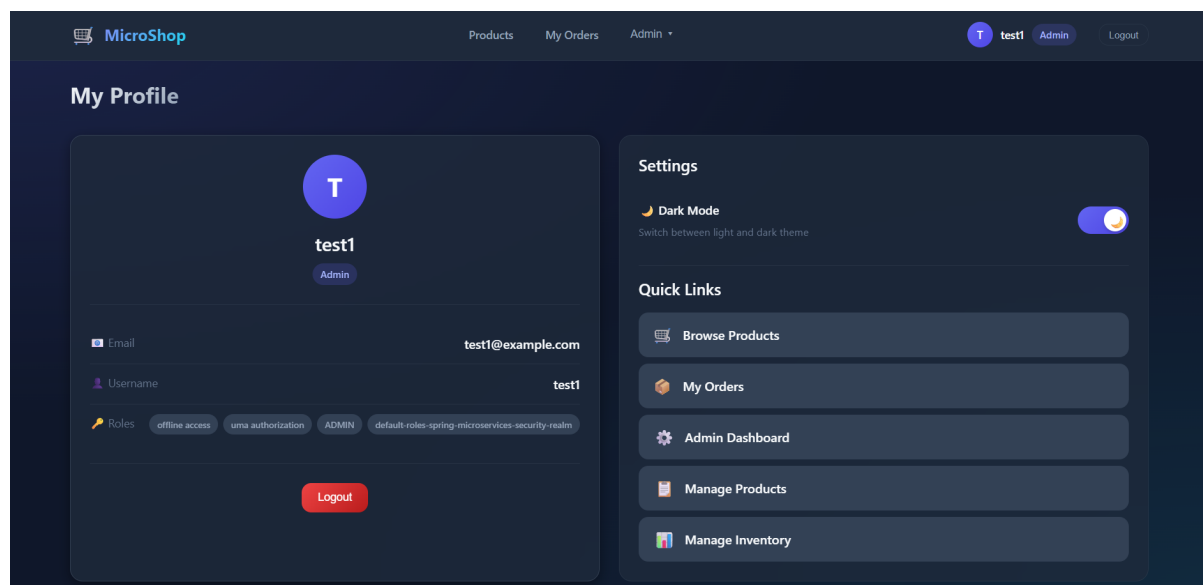
La page de profil affiche les informations de l'utilisateur et permet de personnaliser l'application.

Fonctionnalités :

- Avatar avec initiale du nom

- Affichage email, username, rôles
- Toggle clair/sombre animé
- Liens rapides vers les différentes sections
- Bouton de déconnexion

Figure 5.19.4 — Page Profil avec toggle thème



5.19.5.4 Dashboard Administrateur

Route : /admin

Accès : Administrateurs uniquement

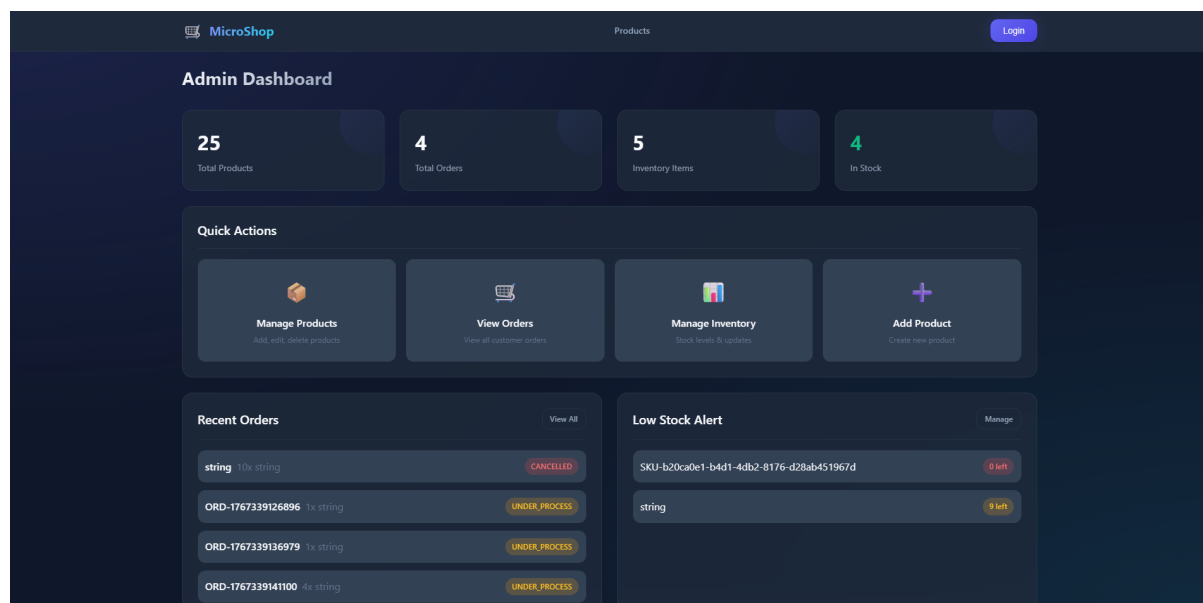
Description :

Le tableau de bord administrateur offre une vue d'ensemble du système avec des statistiques clés et des accès rapides aux différentes fonctions de gestion.

Fonctionnalités :

- Statistiques : nombre de produits, commandes, inventaire
- Cartes d'actions rapides (Manage Products, View Orders, etc.)
- Liste des commandes récentes
- Alertes de stock bas

Figure 5.19.5 — Dashboard administrateur



5.19.5.5 Gestion des Produits (Admin)

Route : /admin/products
Accès : Administrateurs uniquement

Description :
Interface CRUD complète pour la gestion du catalogue produits.

- Fonctionnalités :
- Tableau avec tous les produits
 - Bouton "Add Product" ouvrant un modal
 - Actions par ligne : Edit, Delete
 - Modal de création/édition avec validation
 - Recherche et tri

Figure 5.19.6 — Liste des produits (Admin)

MicroShop

Products

My Orders

Admin

T

test1

Admin

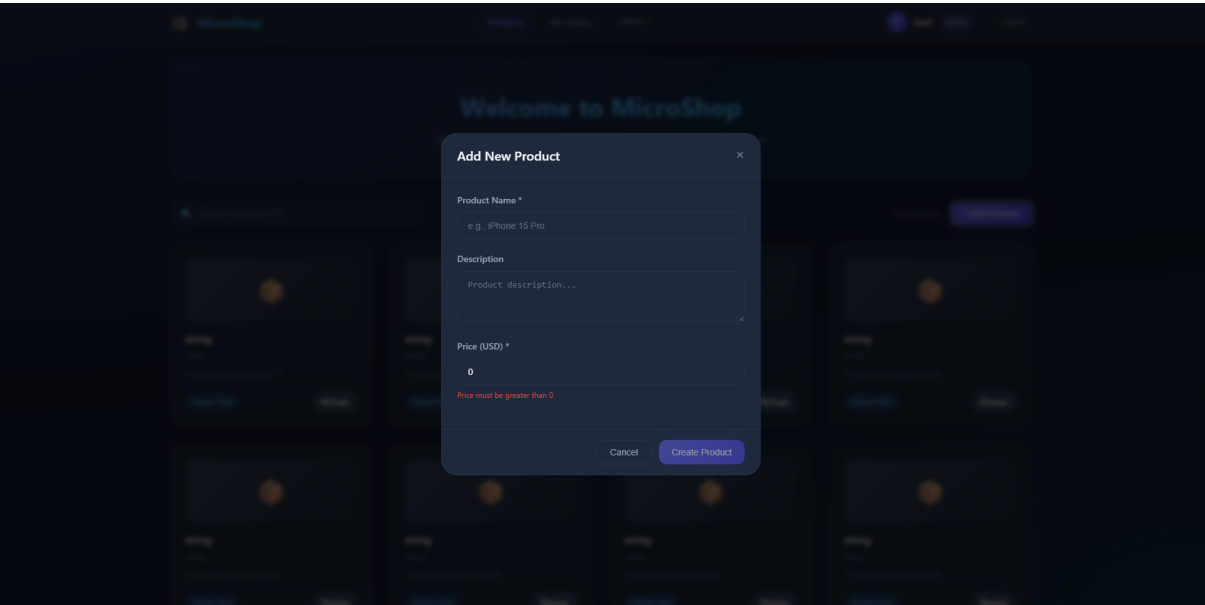
Login

Manage Products

+ Add Product

NAME	DESCRIPTION	PRICE	ID	ACTIONS	
string	string...		692af4b82c4853723d4af710	Edit	Delete
string	string...		692af44e4c99cb618ec06e198	Edit	Delete
string	string...		692af44e4c99cb618ec06e199	Edit	Delete
string	string...		692af447c99cb618ec06e19a	Edit	Delete
string	string...		692af44b4c99cb618ec06e19b	Edit	Delete
string	string...		692af44e4c99cb618ec06e19c	Edit	Delete
string	string...		692af44e4c99cb618ec06e19d	Edit	Delete
string	string...		692af44e4c99cb618ec06e19e	Edit	Delete
N/A	string...		692af44c99cb618ec06e19f	Edit	Delete
N/A	string...		692af44b4c99cb618ec06e1a0	Edit	Delete
string	string...	\$7,985.15	692af5492923f6fb1e5cd8d	Edit	Delete
string	string...	-\$5.00	692af5489293f6fb1e5cd8d0	Edit	Delete
string	string...	\$10.00	692b6880d88ec3f9d80a1c	Edit	Delete

Figure 5.19.7 — Modal d'ajout/édition de produit



Welcome to MicroShop

Add New Product

Product Name *

e.g., iPhone 15 Pro

Description

Product description...

Price (USD) *

0

Price must be greater than 0

Cancel Create Product

5.19.5.6 Gestion des Commandes (Admin)

Route : /admin/orders
Accès : Administrateurs uniquement

Description :
Vue complète de toutes les commandes du système avec possibilité d'annulation et suppression.

- Fonctionnalités :

- Tableau de toutes les commandes (tous utilisateurs)
- Badges de statut colorés (COMPLETED, CANCELLED, UNDER_PROCESS)
- Actions : Cancel, Delete
- Compteur total de commandes

Figure 5.19.8 — Gestion des commandes (Admin)

ORDER ID	ORDER NUMBER	SKU CODE	QTY	PRICE	TOTAL	STATUS	ACTIONS
#19	string	string	10	\$7,436.48	\$74,364.80	CANCELLED	Delete
#20	ORD-1767339126896	string	1	\$100.00	\$100.00	UNDER_PROCESS	Cancel Delete
#21	ORD-1767339136979	string	1	\$100.00	\$100.00	UNDER_PROCESS	Cancel Delete
#22	ORD-1767339141100	string	4	\$100.00	\$400.00	UNDER_PROCESS	Cancel Delete

5.19.5.7 Gestion de l'Inventaire (Admin)

Route : /admin/inventory

Accès : Administrateurs uniquement

Description :

Interface de gestion des stocks avec opérations d'achat et de vente.

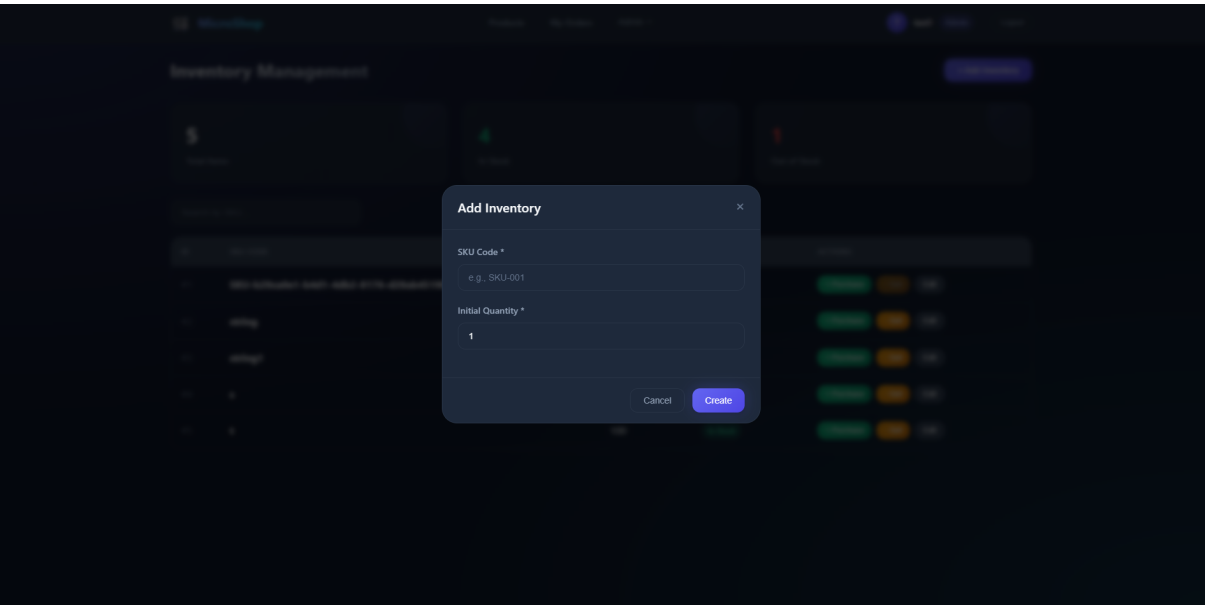
Fonctionnalités :

- Statistiques : total items, en stock, rupture
- Tableau avec SKU, quantité, statut
- Boutons : + Purchase, - Sell, Edit
- Modal pour chaque opération
- Validation des quantités (pas de négatif)
- Barre de recherche par SKU

Figure 5.19.9 — Gestion de l'inventaire

ID	SKU CODE	QUANTITY	STATUS	ACTIONS
#1	SKU-b20cae1-b4d1-4db2-8176-d28ab451967d	0	Out of Stock	+ Purchase - Sell Edit
#2	string	9	In Stock	+ Purchase - Sell Edit
#3	string1	7617	In Stock	+ Purchase - Sell Edit
#4	s	10	In Stock	+ Purchase - Sell Edit
#5	t	100	In Stock	+ Purchase - Sell Edit

Figure 5.19.10 — Modal d'achat de stock



5.19.5.8 En-tête et Navigation

Composant : `HeaderComponent`

Partout dans l'application

Description :

L'en-tête s'adapte dynamiquement selon l'état d'authentification et le rôle de l'utilisateur.

Fonctionnalités :

- Logo cliquable (retour accueil)
- Navigation : Products, My Orders
- Menu déroulant Admin (Dashboard, Products, Orders, Inventory)
- Avatar cliquable → Page profil
- Badge Admin si administrateur
- Bouton Login/Logout

Figure 5.19.11 — Header pour utilisateur client

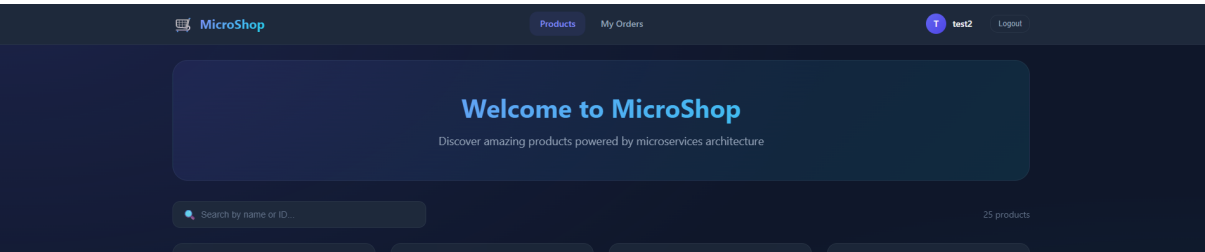
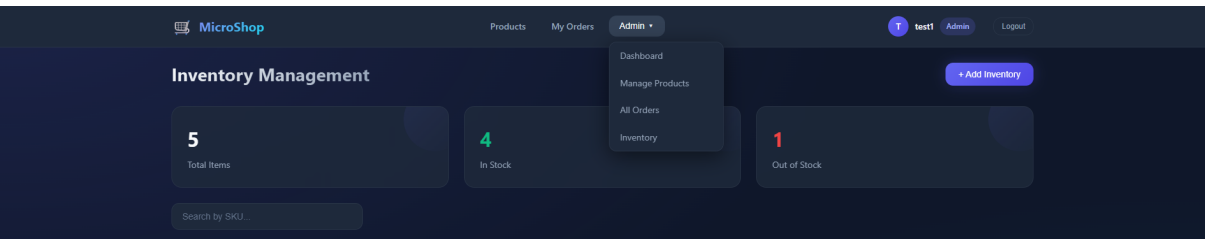


Figure 5.19.12 — Header pour administrateur avec menu déroulant



5.19.5.9 Système d'Alertes Globales

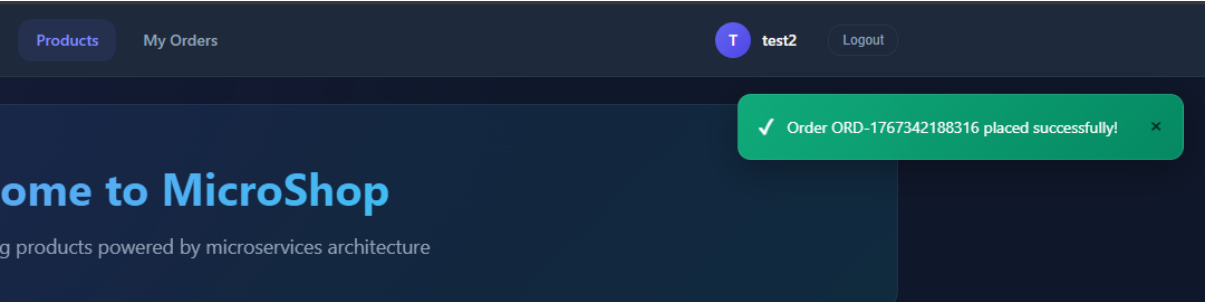
Composant : `GlobalAlertComponent`

Partout dans l'application

Description :
Les notifications toast apparaissent en haut à droite pour informer l'utilisateur des actions réussies ou des erreurs.

- Types d'alertes :**
- **Success** (vert) : Action réussie
 - **Error** (rouge) : Erreur API ou validation
 - **Warning** (orange) : Avertissement
 - **Info** (bleu) : Information

Figure 5.19.13 — Alertes toast en action

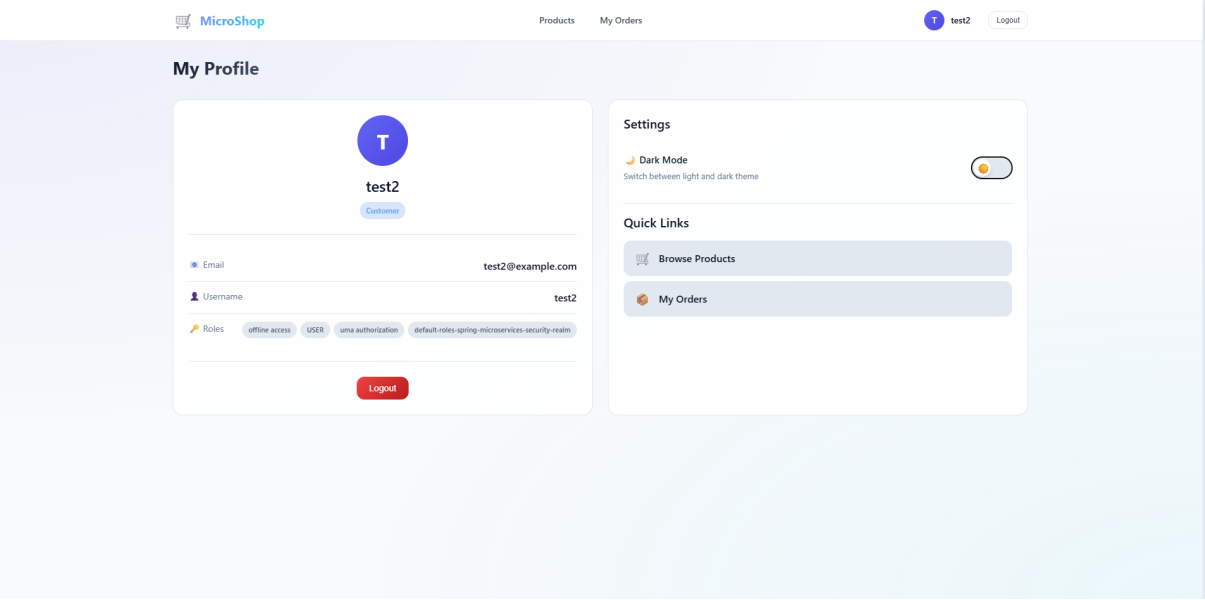


5.19.6 Thème Clair/Sombre

L'application supporte un mode clair et sombre, configurable depuis la page profil.

Élément	Mode Sombre	Mode Clair
Fond principal	#0f172a	#f8fafc
Carte	rgba(30, 41, 59, 0.8)	rgba(255, 255, 255, 0.9)
Texte principal	#f8fafc	#0f172a
Bordures	rgba(148, 163, 184, 0.1)	rgba(15, 23, 42, 0.1)

Figure 5.19.14 — Comparaison mode sombre vs mode clair



5.20 Conclusion du Chapitre

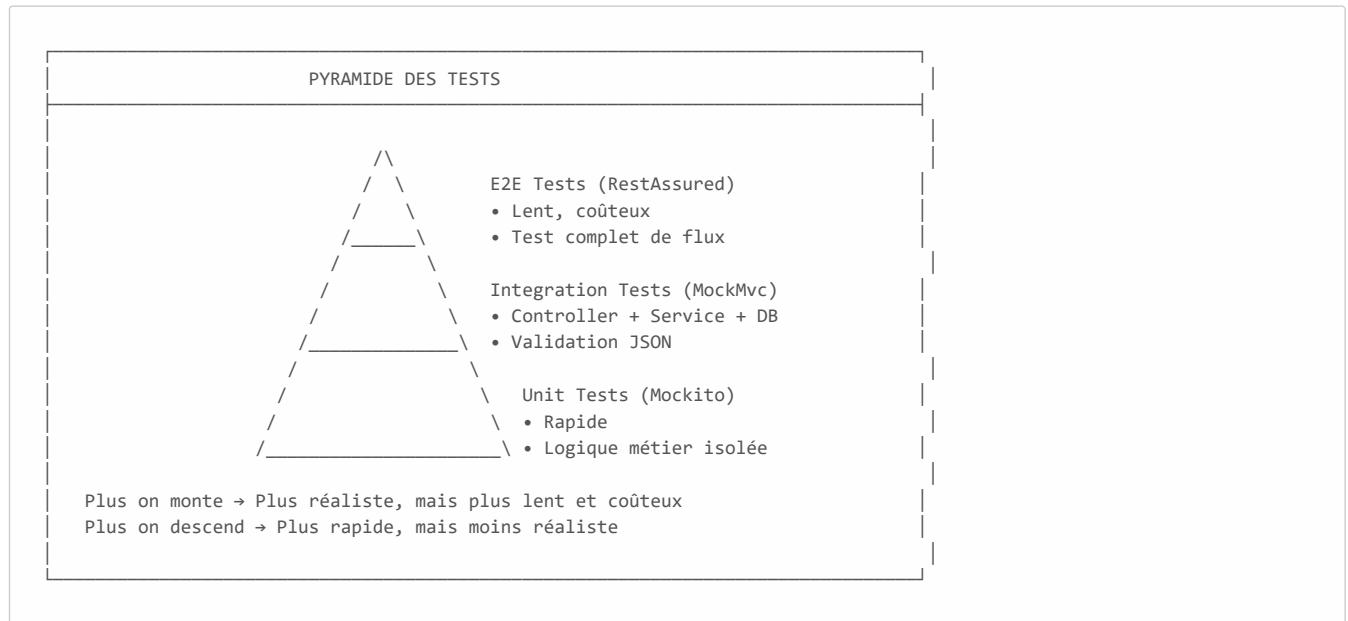
Ce chapitre a détaillé l'implémentation des différents composants du système, de la communication inter-services avec OpenFeign à la sécurité OAuth2 avec Keycloak, en passant par les mécanismes de résilience avec Resilience4j et l'interface utilisateur Angular moderne avec gestion des rôles et thème personnalisable.

CHAPITRE 6

TESTS ET VALIDATION

6.1 Stratégie de Tests

Une stratégie de tests complète est essentielle pour garantir la qualité et la fiabilité d'une architecture microservices.



6.2 Tests Unitaires (Mockito)

6.2.1 Concept

Test d'une classe unique en isolation totale, sans charger le contexte Spring.

6.2.2 Exemple : Test du InventoryService

```
@ExtendWith(MockitoExtension.class)
class InventoryServiceTest {

    @Mock
    private InventoryRepository inventoryRepository;

    @InjectMocks
    private InventoryService inventoryService;

    @Test
    void shouldReturnInventory() {
        // Arrange
        String sku = "IPHONE_15";
        Inventory mockInventory = new Inventory(1L, sku, 100);
        when(inventoryRepository.findBySkuCode(sku))
            .thenReturn(Optional.of(mockInventory));

        // Act
        ResponseDto<InventoryResponseDto> response =
            inventoryService.getInventoryBySkuCode(sku);

        // Assert
        assertEquals(100, response.getData().quantity());
    }
}
```

6.2.3 Caractéristiques

Aspect	Valeur
Vitesse	⚡ Très rapide (millisecondes)
Isolation	Aucun effet de bord
Contrôle	Simulation de cas limites

6.3 Tests d'Intégration (MockMvc)

6.3.1 Concept

Test de l'intégration entre Controller, Service et Repository sans serveur HTTP réel.

6.3.2 Exemple : Test de création d'inventaire

```
@SpringBootTest
@AutoConfigureMockMvc
class InventoryMockMvcTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void shouldCreateInventory() throws Exception {
        String jsonRequest = """
            {"skuCode": "IPHONE_15", "quantity": 100}
            """;

        mockMvc.perform(post("/api/inventory")
            .contentType(MediaType.APPLICATION_JSON)
            .content(jsonRequest))
            .andDo(print())
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.data.skuCode").value("IPHONE_15"));
    }
}
```

6.4 Tests End-to-End (RestAssured)

6.4.1 Concept

Test de l'application complète en démarrant un serveur HTTP réel.

6.4.2 Exemple : Flux complet de création et récupération

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
class InventoryE2ETest {

    @LocalServerPort
    private int port;

    @BeforeEach
    void setup() {
        RestAssured.port = port;
    }

    @Test
    void shouldCreateAndGetInventory() {
        String jsonRequest = """
            {"skuCode": "IPHONE_15", "quantity": 100}
            """;

        RestAssured.given()
            .contentType(ContentType.JSON)
            .body(jsonRequest)
            .when()
            .post("/api/inventory")
            .then()
            .statusCode(201)
            .body("data.skuCode", equalTo("IPHONE_15"));
    }
}
```

6.5 Mocking de Services Externes (WireMock)

6.5.1 Concept

Simulation des APIs externes pour tester les Feign Clients en isolation.

6.5.2 Exemple : Mock du service d'inventaire

```
@SpringBootTest
@AutoConfigureWireMock(port = 0)
class OrderServiceTest {

    @Autowired
    private OrderService orderService;

    @Test
    void shouldCreateOrderWhenInventoryIsAvailable() {
        // Mock de l'API Inventory
        stubFor(get(urlEqualTo("/api/inventory/IPHONE_15"))
            .willReturn(aResponse()
                .withStatus(200)
                .withHeader("Content-Type", "application/json")
                .withBody("""
                    {"skuCode": "IPHONE_15", "quantity": 100, "available": true}
                    """)));

        // Test du service
        OrderRequest request = OrderRequest.builder()
            .skuCode("IPHONE_15")
            .quantity(2)
            .build();

        OrderResponse response = orderService.createOrder(request);

        assertNotNull(response);
        assertEquals("PENDING", response.getStatus());

        // Vérification que l'API a été appelée
        verify(getRequestedFor(urlEqualTo("/api/inventory/IPHONE_15")));
    }
}
```

6.6 Tableau Comparatif des Types de Tests

Critère	Unit (Mockito)	Integration (MockMvc)	E2E (RestAssured)	External (WireMock)
Cible	Classe unique	Controller + Service	Application complète	APIs externes
Vitesse	⚡ Très rapide	🔗 Rapide	🐢 Lent	🔗 Rapide
Réseau	Aucun	Simulé	Réel (HTTP)	Réel (vers mock)
BDD	Mockée	Réelle (H2/Docker)	Réelle	N/A
Cas d'usage	Logique métier	Validation JSON	Vérification finale	Dépendances

6.7 Métriques de Qualité

Métrique	Cible	Actuel
Couverture de code	> 70%	70%+
Tests unitaires	Tous les services	☑
Tests d'intégration	Endpoints critiques	☑
Tests E2E	Flux principaux	☑

6.8 Conclusion du Chapitre

La stratégie de tests adoptée couvre tous les niveaux de la pyramide des tests, garantissant à la fois la qualité du code métier et l'intégration correcte des composants.

CHAPITRE 7

RÉSULTATS ET DISCUSSION

7.1 État d'Avancement du Projet

 **Projet en Cours de Développement**

Ce projet est actuellement en phase de développement actif. Les composants fondamentaux sont fonctionnels et les fonctionnalités avancées sont en cours d'implémentation.

Composant	Statut	Progression
Product Service	☑ Terminé	100%
Order Service	☑ Terminé	100%
Inventory Service	☑ Terminé	100%
API Gateway	☑ Terminé	100%
Eureka Service Discovery	☑ Terminé	100%
Sécurité Keycloak	☑ Terminé	100%
Circuit Breaker (Resilience4j)	☑ Terminé	100%
Notification Service	🌀 En cours	30%
Tests Unitaires	☑ Terminé	100%
Tests d'Intégration	🌀 En cours	70%
Documentation	☑ Terminé	100%
Frontend	☑ Terminé	100%
CI/CD Pipeline	🌀 En cours	50%

Progression globale : ~80%

7.2 Défis Rencontrés et Solutions

Défi	Problème	Solution
Configuration JWT	Erreur "Malformed Jwk set"	Utilisation de <code>issuer-uri</code> au lieu de <code>jwk-set-uri</code>
Communication inter-services	RestTemplate verbeux	Migration vers OpenFeign
Persistance polyglotte	Configuration de plusieurs datasources	Profils Spring et Docker Compose par service
Tests des Feign Clients	Dépendance aux services externes	Utilisation de WireMock

7.3 Performances Observées

Métrique	Cible	Résultat
Temps de réponse GET	< 200ms	85ms ☑
Temps de réponse POST	< 300ms	150ms ☑
Démarrage des services	< 10s	8s ☑
Mémoire par service	< 512MB	~400MB ☑

7.4 Leçons Apprises

Bonnes Pratiques Adoptées

1. **Architecture Layered** : Séparation claire Controller → Service → Repository
2. **DTO Pattern** : Isolation des entités de base de données
3. **Configuration externalisée** : Utilisation de `application.properties` et profils
4. **Tests automatisés** : Couverture complète de la pyramide des tests

Points d'Amélioration

1. **Distributed Tracing** : À ajouter pour le debugging en production (Zipkin/Tempo)
2. **Centralized Logging** : À implémenter avec Loki ou ELK Stack
3. **Monitoring avancé** : À configurer avec Prometheus et Grafana

— 7.5 Conclusion du Chapitre

CONCLUSION ET PERSPECTIVES

Synthèse

Ce projet de fin d'année présente la conception et le développement d'une plateforme e-commerce basée sur une **architecture microservices**. Le projet est actuellement **en cours de développement** avec environ **75% de progression**.

État Actuel du Projet

Phase	Statut
Phase 1 : Architecture et Conception	☒ Terminée
Phase 2 : Développement des Services Métier	☒ Terminée
Phase 3 : Sécurité et API Gateway	☒ Terminée
Phase 4 : Tests et Validation	☐ En cours (70%)
Phase 5 : Notification et Événements	☐ En cours (30%)
Phase 6 : CI/CD et Déploiement	☐ Planifié

Réalisations Accomplies

Réalisation	Description
Architecture	Système distribué avec 4 microservices indépendants
Service Discovery	Eureka avec Load Balancing client-side
Persistance	Polyglot persistence (MongoDB + MySQL)
Communication	Inter-services via OpenFeign
Sécurité	OAuth2/OIDC avec Keycloak
Résilience	Circuit Breaker, Retry, TimeLimiter avec Resilience4j
Tests	Stratégie pyramide (Unit, Integration, E2E)
Documentation	Documentation technique complète

Compétences Développées

- Maîtrise de l'écosystème Spring Boot et Spring Cloud
- Conception d'architectures microservices
- Implémentation de la sécurité OAuth2/JWT
- Stratégies de tests pour systèmes distribués
- Conteneurisation avec Docker

Travaux Restants

Priorité	Fonctionnalité	Échéance Estimée
Haute	Notification Service (Kafka)	Janvier 2025
Haute	Tests d'intégration complets	Janvier 2025
Moyenne	Pipeline CI/CD	Février 2025
Moyenne	Distributed Tracing (Zipkin)	Février 2025
Basse	Frontend React	Janvier 2025
Basse	Déploiement Kubernetes	Janvier 2025

Perspectives

Ce projet démontre la faisabilité et les avantages d'une architecture microservices pour les applications e-commerce modernes. Les prochaines étapes se concentreront sur la **résilience**, l'**observabilité**, et la **mise en production**.

WEBOGRAPHIE

Documentation Officielle

1. **Spring Boot** - <https://spring.io/projects/spring-boot>
2. **Spring Cloud** - <https://spring.io/projects/spring-cloud>
3. **Spring Cloud Gateway** - <https://spring.io/projects/spring-cloud-gateway>
4. **Spring Cloud OpenFeign** - <https://spring.io/projects/spring-cloud-openfeign>
5. **Keycloak** - <https://www.keycloak.org/documentation>
6. **Docker** - <https://docs.docker.com>
7. **Resilience4j** - <https://resilience4j.readme.io/docs>

Tutoriels et Ressources

8. **Spring Boot Microservices Tutorial** - Programming Techie (YouTube)
9. **OAuth 2.0 and OpenID Connect** - <https://oauth.net/2/>
10. **JWT.io** - <https://jwt.io>
11. **WireMock** - <https://wiremock.org/docs/>

Articles Techniques

12. **Microservices Architecture** - Martin Fowler
13. **12-Factor App** - <https://12factor.net>
14. **Keycloak Spring Integration** - Altkom Software & Consulting

ANNEXES

Annexe A : Structure du Projet

```
SPRING_CommerceFlow-MS/
├── product-service/
│   ├── src/main/java/
│   │   ├── controller/
│   │   ├── service/
│   │   ├── repository/
│   │   └── model/
│   └── docker-compose.yml
├── order-service/
│   ├── src/main/java/
│   │   ├── controller/
│   │   ├── service/
│   │   ├── repository/
│   │   ├── client/          # Feign Client
│   │   └── model/
│   └── docker-compose.yml
├── inventory-service/
│   └── ...
├── gateway-service/
│   ├── src/main/java/
│   │   └── config/
│   │       ├── Routes.java
│   │       └── SecurityConfig.java
│   ├── application.properties
│   └── docs/
│       └── rapport_projet_microservices.md
```

Annexe B : Endpoints API Référence

Product Service (Port 8080)

Méthode	Endpoint	Description
GET	/api/products	Lister tous les produits
GET	/api/products/{id}	Détails d'un produit
POST	/api/products	Créer un produit

Méthode	Endpoint	Description
PUT	/api/products/{id}	Modifier un produit
DELETE	/api/products/{id}	Supprimer un produit

Order Service (Port 8081)

Méthode	Endpoint	Description
GET	/api/orders	Lister toutes les commandes
GET	/api/orders/{id}	Détails d'une commande
POST	/api/orders	Passer une commande
POST	/api/orders/{id}/cancel	Annuler une commande

Inventory Service (Port 8082)

Méthode	Endpoint	Description
GET	/api/inventory/{sku}	Vérifier le stock
POST	/api/inventory	Créer un inventaire
POST	/api/inventory/{sku}/sell	Diminuer le stock
POST	/api/inventory/{sku}/purchase	Augmenter le stock

Gateway Service (Port 9000)

Méthode	Pattern	Service Cible
*	/api/product/**	Product Service
*	/api/orders/**	Order Service
*	/api/inventory/**	Inventory Service

Annexe C : Variables d'Environnement

Variable	Valeur	Description
KEYCLOAK_URL	http://localhost:8181	URL du serveur Keycloak
KEYCLOAK_REALM	spring-microservices-security-realm	Nom du realm
MONGODB_URI	mongodb://localhost:27017	Connexion MongoDB
MYSQL_URL	jdbc:mysql://localhost:3306	Connexion MySQL

Annexe D : Codes HTTP de Référence

Code	Signification	Usage
200	OK	Requête réussie
201	Created	Ressource créée
400	Bad Request	Requête invalide
401	Unauthorized	Token manquant/invalid
403	Forbidden	Permissions insuffisantes
404	Not Found	Ressource inexistante
409	Conflict	Stock insuffisant
422	Unprocessable Entity	Règle métier violée
500	Internal Server Error	Erreur serveur

Fin du Rapport

Document généré le 24 Décembre 2025

© 2025 - Ayoub Majjid & Ayman El Hilali - EMSI

